

WBL Project Report

By

Suber Abdi w2069210

Gabriel Lopez Narvaez w2090932

Nicholas Nicolaou w2047568

Charanjot Sidhu w2047567

Caleb Suleaudu w1910808

Victoria Veprek w2040013

Table of Contents

Contribution Table	4
AI Declaration	4
Abstract.....	5
Introduction.....	5
Section 1: Theory	7
1.1 Motor Theory	7
1.2 Stepper Motor Types & Control.....	8
1.3 Sensors & Electromagnetic Waves	9
1.4 Noise & Interference	10
1.5 Navigation.....	11
1.6 Mapping and Path Analysis	11
Left Wall Only	11
Lee’ s Algorithm	12
Dijkstra’ s Algorithm	12
1.7 Batteries & Load	13
1.8 Driving Large Currents	15
1.9 The MBED Platform.....	16
1.10 Product Life Cycle Stages	17
1.11 Risk and possible mitigation for product life cycle stages	17
Section 2: Broader Issues	19
2.1 Industrial/commercial application of robots like the micro-mouse	19
2.2 Environment considerations and suggested mitigations	19
2.3 Sustainability considerations and suggested mitigations	20
2.4 Social impact of similar robot to the micro-mouse	20
2.5 Commercial considerations, Intellectual property rights and licensing	21
2.6 Legal issues, regulation and compliance for robots like the Micro-Mouse	21
2.7 Identified risks and suggested mitigations for the micro-mouse robot.....	22
2.8 Professional codes of conduct, ethics, data security and privacy	22
2.9 Health and Safety	22
Section 3: Operating Requirements.....	23

3.1 System and Operating Environment	23
3.2 Functional Requirements	24
3.3 Mechanical Requirements	25
Section 4: Specification and Costs of a Micro-Mouse Robot	25
4.1 Component Specifications and Justification	25
4.2 Budget and Costing	27
4.3 Sustainability and Environmental Considerations	28
4.4 Regulatory Compliance	28
Section 5: Hardware Design & Mechanics	29
5.1 Chassis Dimensions, Layout & Design	29
5.2 Power Distribution Circuits	29
5.3 Motor Driver Circuits & Motors	29
5.4 Sensor Configuration and Sensor Circuits	29
5.5 Micro-Controller Board Circuitry	30
5.6 Power Source	30
Section 6: Software Design & Logic	31
6.1 Overall System Discussion	31
6.2 Software Structure	32
6.3 Sensor Logic used for Navigation	33
6.4 Post Processing of Sensory Information	35
6.5 State Machine & Navigation Implementation	36
6.6 Mapping and Path Analysis Solutions	39
6.7 Power Management and Efficiencies	41
Section 7: Implementation of Hardware	42
Section 8: Testing	43
8.1 Design for Test Approaches Adopted	43
8.2 Testing Methodology	44
8.3 Testing of Sensor Circuits & Outcomes	45
8.5 Testing Micro-Mouse State Machine & Timing	47
8.6 Testing of the Overall Micro-Mouse & Outcomes	48
Section 9: Teamwork, Planning and Leadership	50

Conclusion	54
C.1 Achievements and Success.....	54
C.2 Component Choice and Future Changes	54
C.3 Societal Impact and PLC	55
C.4 Group Management	56
C.5 Impact of Mentoring Sessions	56
References	57
Appendix A: Schematics	59
Appendix B: Full Code	61

Contribution Table

Member	Primary Role/ Tasks	Contribution Rate
Suber	Software/Mechanical - creating the structure of the code and helping with the assembly of the micro mouse	20
Gabriel	Hardware - creating motor driver circuits, testing for torque, and testing the circuits for shorts	20
Nicholas	Software - Use of Keil studios, updating GitHub and Testing code on MBED	20
Charanjot	Software - Use of Keil studios, updating GitHub and Testing code on MBED	15
Caleb	Hardware - Made ultrasonic sensor and helping with hardware testing	10
Victoria	Hardware/Mechanical - making the IR-sensors and testing them, also aiding with the assembly of the micro mouse	15

AI Declaration

This report/ presentation has been prepared based on my own work. Where other published and unpublished source materials have been used, these have been acknowledged in the references. In preparing this project, we used the following generative AI tools:

Tools	How it was used
Gemini/GPT	Explaining Concepts, Suggesting Books/ Reference Material on Theory

Claude

Generating Code Suggestions
(conversion to OS6)

Abstract

This report explains in detail the theory and technical background behind a maze solving robot which needs to navigate a maze that is 8x8, the operating requirements and ability to produce a well-planned final product as well as the ability to test the strategy. These details are discussed in different sections, analysing the complexity of an embedded system that involves C++ code, precise control over stepper motors, implementation of a maze mapping algorithm such as Lee's algorithm, wall detection capability through IR sensors. Moreover, the testing of hardware and software that occurred to make sure the robot would withstand a real-world environment and stay stable during real usage.

Furthermore, the report explores the broader issues that complex robots that have similar features to a micro mouse, which includes worldly impacts, practical applications and codes of conducts. In addition to this, it has been examined how our group will collaborate, delegated roles and responsibilities, and who leads in certain project aspects.

Introduction

Competitions involving micro mouse have gone on for years such as the IEEE competition, producing events with high benchmarks in autonomous robotics that require the combined effort of algorithmic logic, mechatronics that functions at high speeds, precise sensors data collection and fast processing of real-time events.

The main challenge consists of navigating an 8x8 maze with cells approximately the size of 16 x 16 cm. It is required for the mouse to travel from a known starting point to the end of a maze without prior knowledge of its layout whilst taking into account any operating requirements, and physical constraints or external environments that may affect runs. The secondary challenge is the mouse abilities to process the real-world data is collected and exhibit a suitable reaction to achieve it's intended purpose.

This report provided a breakdown of the project including theoretical and technical background, to design and testing strategies, to in detail descriptions of the software and hardware, and more.

- Section 1 examines the underlying engineering principles and theory needed to understand how to build a robot, understanding the tools required or available as well as concepts that need to be understood to know what can have an impact on micro mouse performance and how to counter it.

- Section 2 delves into the potential application of the different parts/technologies need to make a micro mouse. In addition to this section 2 goes through the real-world impact an autonomous robot can have and the codes of conduct necessary to make a project like this.
- Section 3 explores the operating requirements needed for a micro mouse to function including the external factors it needs to take into, the features it needs to be able to function in the maze and how mechanical factors such as weight or placement are taken into account for the assembly of the micro mouse.
- Section 4 elaborates on the specification and required components that are needed for the micro mouse as well, this also includes the cost, number and supplier of components.
- Section 5 examines the hardware design, mechanics and chosen layout the micro mouse needs to be able to move and provide the micro controller with the information it needs to decide the action required. In addition to this, it also discusses how the parts the micro controller uses to function.
- Section 6 presents the software behind the hardware, the engine that will decide what the micro mouse will after receive sensor data and processing it for the optimal outcome. This section talks in detail the theory, structure and features of MBED OS needed as well as certain techniques used to account for hardware constraints.
- Section 7 explores how the hardware is implemented such as its layout and how different parts of the mouse have been split into sub-circuits on different Vero board
- Section 8 analyses the testing strategies used to evaluate the current condition of the parts of the micro mouse, the software and how it functions with the hardware as well the test choices approached
- Section 9 explains how the team managed time, broke down the project into smaller tasks and assigned tasks, planned the work for the WBL project, documented meeting, and more. This is shown through a variety of ways such as minutes of meeting, logbooks and Gantt charts.

Section 1: Theory

1.1 Motor Theory

An electric motor is a device/ component which converts electrical energy into mechanical energy using magnetic fields and conductors. One type of electric motor is a brushed dc motor which has through two terminals that have no negative or positive polarity at first but switch the polarity to achieve rotation, the control of its speed is done by changing the supply voltage. The method of controlling speed means this type of motor can be controlled by an MCU which can control the analog switch to change the 2 terminal wirings. [1]

Another type of electric motor is a brushless motor which has a rotor on the inside that is made of magnetic material whilst having a stator on the outside which generates a rotating magnetic field. The rotor is made of a closed circuits of conductors which allows it to generate its own induced current because of the stator's magnetic field. A similarity between these types of motor is that they are capable of rotating under magnetic forces. [1]



Figure 1: Image of a brushed DC motor [1]

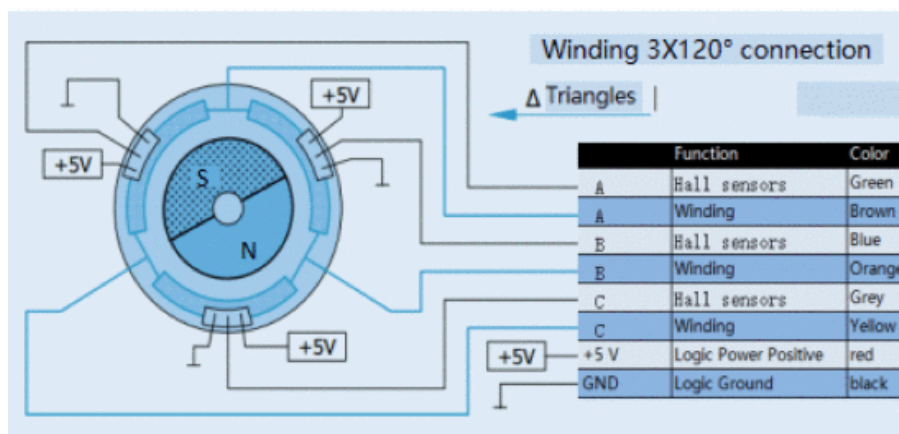


Figure 2: Wiring / Layout of a brushless DC motor [1]

1.2 Stepper Motor Types & Control

A stepper motor is a type of brushed DC motor which works by using a wire coil (the rotor) to rotate inside the stator (a fixed magnet housing). The rotor is an electromagnet that uses magnetic attraction and repulsion to achieve movement. The motor also uses a combination of brushes known as electrical contact and the quick alternating change of the direction of current using a commutator (electrical switch). This creates the conditions needed for continuous spinning of the rotor coil. [2]

There are different types of stepper motors which includes bipolar stepper motors and hybrid stepper motors. Bipolar stepper motors work by using an on-board driver with a H-bridge circuit, this allows the current flow to be reverse through the phases, all the coils in the motor are moved by alternating the polarity whilst energising the phases. These types of motors in practicality are better since they use more of the coil windings than a standard unipolar stepper motor. [2]

Hybrid stepper motors are similar but generally have teeth around the outer parts of the magnet rotor. Moreover, they are more precise as they can use techniques such as half-stepping and micro-stepping. They are also better than standard stepper motors as they have a higher torque to size ratio and higher output speeds. [2]



Figure 3: Image of a stepper motor from RS components [2]

1.3 Sensors & Electromagnetic Waves

Sensors are important for devices to have as they are the way for them to take in real world data that can be converted and processed for evaluation. The most important things that evaluate how good a sensor may be is its ability to receive information as precise and accurately as possible even if there's some noise and interference. [3]

There are different types of sensors which includes analog and digital sensors. Analog sensors are devices that produce a constant output signal that is proportional to the physical quantity (e.g. light, sound) it measured, digital sensors are devices that output 1 or 0 signals [3]. For robots like a micro mouse, analog sensors similar to infra-red sensors and digital sensors such as an ultrasonic sensor are necessary to detect the walls of a maze. Infra-red sensors consist of an LED (emitter) and a phototransistor (receiver), the emitter will send electromagnetic waves between 850nm and 950nm through the air until they hit an object that will either reflect or absorb it [3] [4]. If the wave reflects to the receiver, this can be used by an ADC convertor in a microcontroller to convert the current into a digital value usable by software [4].



Figure 4: An image of an IR emitter (OP165D LED) [5]



Figure 5 : An image of an IR receiver (IS471F) [6]

Ultrasonic sensors like the HC-SR04 uses soundwave like a sonic burst which will bounce back to the receiving part of the sensors, with a MCU and a program it is possible to measure the time interval it takes to receive an echo to calculate the distance using the equation, $\text{distance} = (\text{Time } (\mu\text{s}) \times 0.0343)/2$, an ultrasonic sensor only cares about the object and not its characteristics, this means its less affected by the wall's colour and transparency [7].



Figure 6: Image of an ultrasonic sensor (HC-SR04) [7]

1.4 Noise & Interference

Noise are random and unwanted factors that can affect a signal through electrical or physical disturbance, these factors come from the electrical components themselves which can affect the reliability of sensor or any electrical components that involves a signal. For example, having sensors to high current components like stepper motors [3]. Interference is an unwanted signal from an external source that is more structured or periodic. Examples of this include ambient and background light that can affect sensor that rely on lighting such as IR sensors, this can also be large voltage spikes. [3]

Solutions to this must be added into the design layout of the micro mouse, sensor circuit or software. These solutions include physical shielding and barriers to block ambient light and make sure the sensor only receives the light reflected. Decoupling capacitors close to the power pins of the sensors to suppress voltage spikes and high frequency noise [2]. There also solution to be done through software design such modulation where the sensors is turned on off in a duty cycle so that they are less likely to interfere with each other [1] [4].

1.5 Navigation

Robots like micro mouse must navigate through a maze moving forwards until a junction/ turn is reached. When starting out the mouse needs to be aware of its orientation so that it knows as to what part of the maze it is heading. As seen in the figure below the different types of intersections possible the mouse must have the ability to understand, identify and record what type of intersection it has reached and take that information and use it in a maze mapping algorithm. Priority of direction is important since when a T-junction is reached the mouse needs to know if it should turn left or right. [8]

To be successful in navigation with little to no errors the mouse needs steering control, the ability to stay aligned in the maze to stop collision during turns. In addition to this the mouse needs to know its orientation and which way it is pointing so that it will not accidentally go in the inverse direction when simulating the maze in it program.

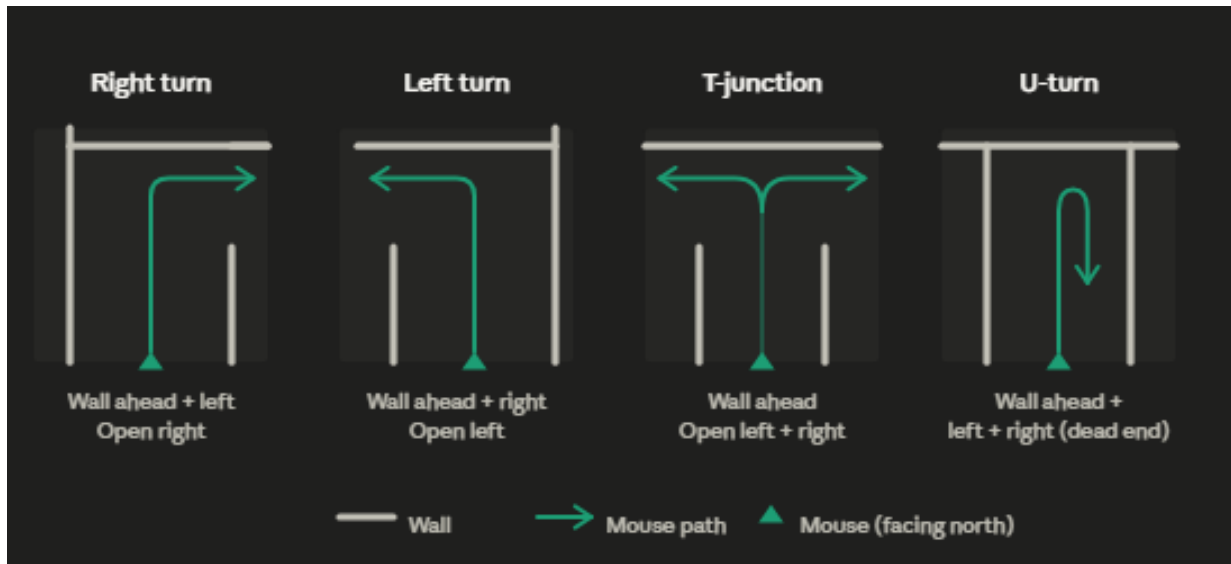


Figure 7: A drawing from draw.io displaying all the different intersections a micro mouse could come across

1.6 Mapping and Path Analysis

Left Wall Only

The left wall only algorithm is when the wall follows only the wall to its left because in theory if there is no loop in the maze, the mouse will inevitably find the end point it is functional required to reach. However, this algorithm has a huge weakness when it comes to time constraints as it will have little to no chance of following an optimal path. Furthermore, it results in failure or infinite looping around the maze if the goal is separated in a “island-like manor” (path with no wall connected to it). [9]

Lee's Algorithm

Lee's algorithm also known as the flood fill algorithm is based on BFS (breadth-first search) where it adds a numerical value to maze to find the shortest path. The end point is started at 0 and the cells adjacent to it are given a value of +1 to the previous value given, this repeats until and the cells are assigned value, in a program this is done using loops. [8]

The robot itself will follow through the cells in descending order following forwards as a priority and using a secondary predesignated priority (such as left if unable to move forwards) when hit with a wall in front. If the robot hits a dead-end it will recalculate all values of the cells but taking into account the wall it hit, it will decide that it won't be able to skip over the wall and give the cell adjacent to it a value until it goes around the wall into different cells first. This entire process is managed by the MCU in real-time whilst using sensor data. [8]

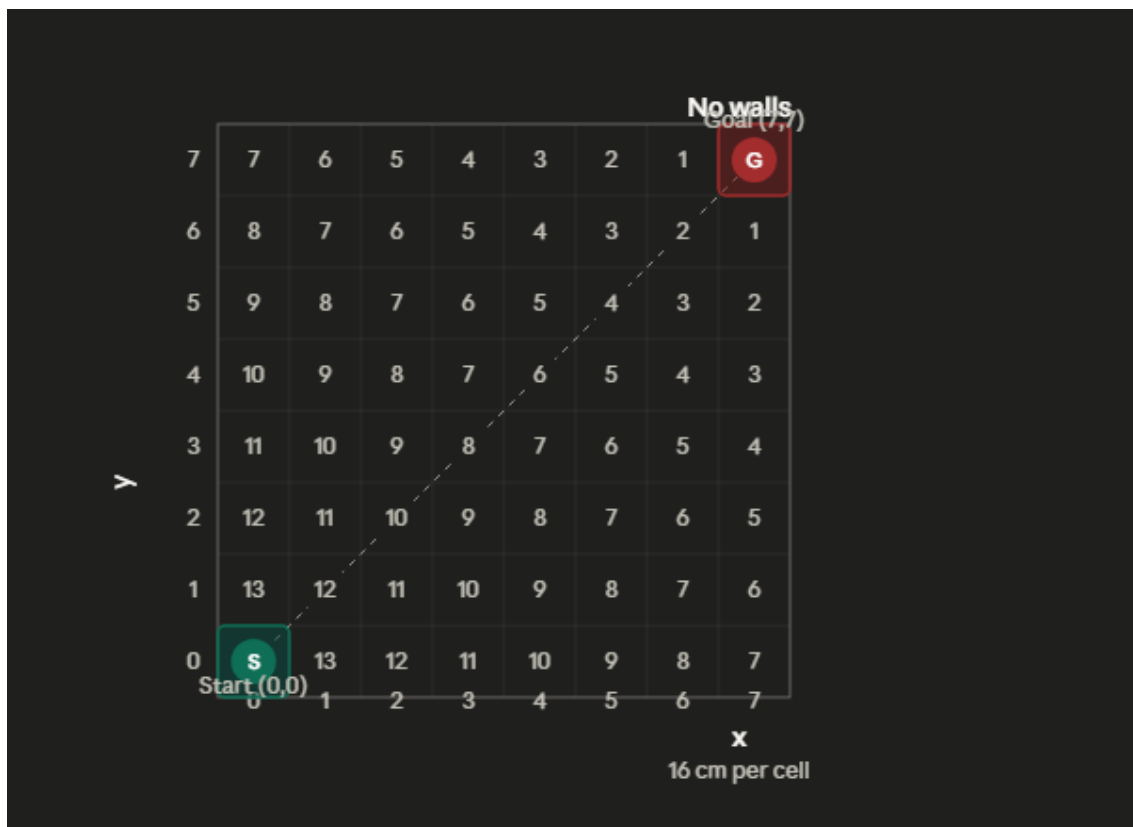


Figure 8: Example of Lee's algorithm, made on draw.io

Dijkstra's Algorithm

Dijkstra's algorithm is a more advanced algorithm than flood fill as it allows the system to account and assign different for different maneuvers. For example, a 90-degree turn is slower than moving through a cell, so moving straight it assigned a value of 1 whilst deciding to do a turn will cost a high value (e.g. 2 or more). It is a weighted search which needs more memory and prioritises a straight path. [10]

1.7 Batteries & Load

A battery works by transforming electric energy from chemical energy using an electrochemical redox (oxidation-reduction) reaction. A battery consists of one or more cells that are connected in different ways which has an impact on the output voltage and capacity. The cells contain an anode (negative electrode) which is oxidized during the reaction and gives electrons to the external circuit as well as a cathode (positive electrode) that does the opposite. [11]

Batteries come in different size, shapes and abilities which has a huge effect on its practical applications, for example devices that are small cannot use a heavy or high heat generating battery as it will cause system failure, these differences can be seen in the table below. A maze moving robot needs a portable battery that will generate 12 volts with a high amount of current (load) whilst having a low weight so that there is minimal to no impact on movement.

When batteries are placed in series in a holder their total voltage is combined and more current (load) can be drawn from them. The estimated total power draw of the university student robot would be roughly near 1 amp; a standard battery holder with normal battery like a Duracell from a supermarket would provide enough; 2 amps [12]. Using the capacity of 2500mAh and total average load of 1A, a standard 8 x AA battery holder would last roughly two hours without using any techniques or power saving design choices. However, due to practical choices like duty cycling or the level of torque needed the battery life of a robot can change significantly.

Type of Batteries	Advantages	Disadvantages
Lithium-Ion	Higher energy density than other types of batteries, can supply enough current for motors, lightweight and long lifespan for a battery [11]	Safety risk due to overheating so requires protection circuits and degrades quicker overtime compared to other batteries [11]
Nickel-Metal Hydride (NiMH)	Eco-friendly and causing less environmental damage, safer to handle [11]	Less durable to heat and a shorter lifespan than batteries like lithium-ion [11]



Figure 9: Battery holder from PI Hut that can hold up to 8 batteries with 12V output with up to 2-amp peak current draw [12]



Figure 10: Example of batteries usable in a micro mouse

A maze solving robot not only requires its own power supply, but it also must have a way to provide different levels of voltage to different components such as the microcontroller needing 5 V whilst the motors need 12 V. This is typically done using a voltage regulator such as the LM7805 to take in a most likely unstable input voltage and give out a stable output voltage, it also outputs a current of 1.5 A which is enough for a micro mouse to power all of its components [14]. A voltage regulator requires a heat sink or a form of cooling system as its operating temperature (T_j) rests between 0°C to 125°C, it also requires two capacitors to stop (0.33 μ F capacitor next to the input and a 0.1 μ F capacitor next to output) to stop noise and interference [14].

1.8 Driving Large Currents

To drive large amounts of current to the stepper motors it's necessary to have a motor driver circuit which can handle which requires two components, the L297 stepper motor controller and the ULN2003A.

The L297 is a specialised integrated circuit that can function as a pattern generator which can manually calculate which coil to turn on and off, generating a four phase drive signals for bipolar (two phase) and unipolar (four phase) stepper motors. To function the device itself only needs an input signal into its clock, direction and mode pins. It has different phase sequences which can produce normal, half-step and wave modes, great for controlled movements. The L297 is intended to be used in Darlington arrays which are circuits that handle high voltage and currents. [15]

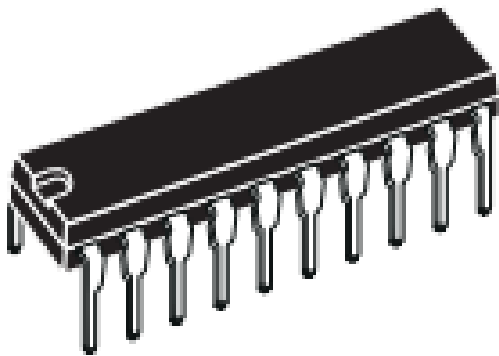


Figure 11: Image of a LP297 from its datasheet [15]

However, the L297 cannot do anything on its own as is just provides signals necessary to control the motor, that responsibility passes to the ULN2003A which are high voltage and current Darlington transistor arrays which current rating is of a signal pair can handle up to 500 mA, the ULN2003A has seven. A UIN2003A has its own flyback diodes which are components that stop voltage from travelling back during spikes. [16]

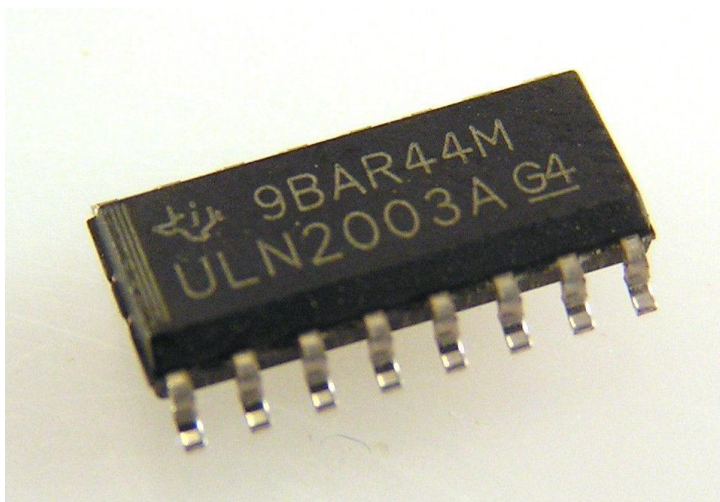


Figure 12: An image of a ULN2003A component [17]

1.9 The MBED Platform

MBED OS which is an open-source operating system that needed to use an ARM Cortex-M microcontroller such as the MBED LPC1768 provides an IDE used for programming MBED boards known as Keil Studio, the library provided is MBED.h programmable with the C++ language. The operating system helps supports most Arm Cortex-M based devices, allowing for quick prototyping on low-cost development boards through simple USB drag and drop programming without the need for expensive debug hardware. There is also driver support a standard MCU peripherals is included in MBED OS. This includes BusIO, PWM, I2C, SPI and serial communications, and digital and analog IO, hardware interrupts. [18]

<https://os.mbed.com/mbed-os/>

A development board like the MBED LPC1768 is a suitable choice it has several GPIO pins, some specialised such AnalogIn (inbuilt ADC) pins, PWMOut (for precise control of motors) as well as high clock speed to do. [19]

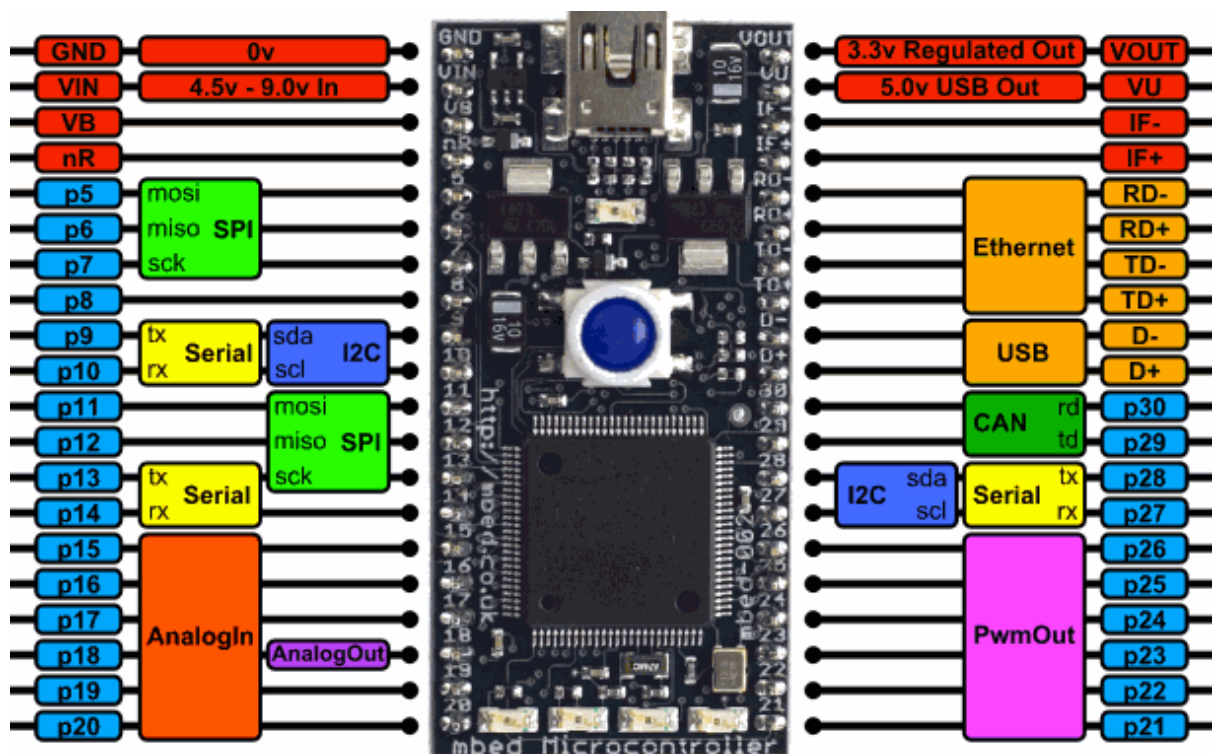


Figure 13: Layout of the MBED LPC1768 [19]

1.10 Product Life Cycle Stages

The micro mouse goes through several stages during its project lifecycle to be able to be made into a final system that fulfils all system, mechanical and functional requirements. Firstly, the micro mouse goes through system design and project planning so components can be decided, factors such as risk, environmental and external factors are taken into account. To do this, weekly meetings and online communication occur to update, plan progress and make alteration to future progress.

The micro mouse itself has its own development stages as there are separate circuits that are a part of the whole system. Firstly, the entire circuit of the robot is split into separate sub-circuits and prototyped on breadboard before being moved onto Vero board. This includes the two IR sensors on separate boards, one Veroboard that contains both motor driver boards, one Veroboard with the MBED and another board that has the power rails and voltage regulator circuit.

At the same time software is produced in parallel, shared via GitHub and tested using MBED simulator. Different structures and layout of a program and made, as well as testing the different features provided by Keil Studio and MBED.h. In addition to this, different algorithmic approaches were discussed including Lee's algorithm and Dijkstra's.

The final stage involves the integration of all subcircuits, connecting the wheels, motor and the circuitry to a selected chassis. During the stages extensive documentation was made such as a Gantt chart to visualise the project timelines, milestones and deadline. This helped ensure that the separate parts of the micro mouse would be made into an autonomous system before the project's deadline

1.11 Risk and possible mitigation for product life cycle stages

There are a multitude of risks that can occur during the product lifecycle including component failure, power loss, program errors, mechanical/assembly faults, group and group management. To manage risks, they were evaluated on the chance that they would occur, how badly it could affect the project and how it could be prevented. However, issues would still mostly likely occur, and it is important to be able to deal with it at the time.

Risk Description	Category	Probability	Impact	Mitigation Strategy
Component Failure (e.g., damaged ULN2003)	Technical	High	Medium	Keep spare components and test them before any long or important usage
LPC1768 Issues (due to motor noise)	Technical	Medium	High	Implement decoupling capacitors and separate the 5V logic rail from the 12V motor rail using the LM7805
Software Deadlock (Maze logic loop)	Technical	Medium	High	Use Timers in MBED OS to reset the MCU if the code deadlocks; implement loop-break conditions in Lee's Algorithm
Team Member Absence	Project	Medium	Medium	Use GitHub, OneDrive and WhatsApp to collaborate so that the project remains accessible to all
Obstacle Collision	Mechanical	High	High	Program IR/US sensors for automatic stop if an object is too close
Power Depletion/ Faulty Batteries	Mechanical	Low	Low	Use multi meter to check the voltage output of the battery before usage, have spares at all times

Section 2: Broader Issues

2.1 Industrial/commercial application of robots like the micro-mouse

The main purpose of a micro-mouse is to solve a maze as quickly as possible [20]. It achieves this by sensing, mapping, and decision-making [21][22]. A micro-mouse itself would never be used in an industrial setting, but the technology could enable it. The technology that a micro-mouse uses could be used in three key areas of industry.

A shipping company could use this path-finding technology to find the shortest path to pick up specific items on a shelf and continuously update routes for efficiency. This would basically be a large-scale, real-world version of a micro mouse, but with more complexity [24]. The sensing technology of a micro-mouse could also benefit an automated guided vehicle. They would use sensors to detect their surroundings and optimise routes to reduce delivery time [23]. These sensors can be used to detect the surroundings, potentially reducing the risk of falls. Modern AGVs have evolved in recent years into autonomous mobile robots that behave more like a micro-mouse, building maps, making real-time decisions, and adapting to changing situations [23]. So, things like the road being closed due to a marathon.

Small autonomous robots, like a micro-mouse, could serve a useful purpose in inventory scanning sections moving through tight warehouse aisles where humans can't fit and checking stock levels [24]. These micro-mouse styled robots would navigate structured environments and move productively without human control [21].

2.2 Environment considerations and suggested mitigations

A micro-mouse has very little impact on the environment. It's a small robot, so its direct impact is minimal. There are environmental benefits of a micro-mouse, such as low energy use, it runs on small batteries and uses very little electricity. The function of a micro-mouse is to always find the shortest path to the maze's end, which represents energy-efficient behaviour [21].

In our own course, we are taking part in the micro-mouse project. By taking part in this project, we are learning to build robots more easily and efficiently, using the least power possible to reach the goal as quickly as possible.

There are some downsides that have a small effect on a micro-mouse, such as electronic waste [25]. When building our own micro-mouse and some components, such as a heat sink, capacitor, batteries, and Veroboard, we found that, during desoldering or testing, these parts were burnt or damaged and were unusable.

A micro-mouse has little effect on the environment due to its size. Battery use and electronic waste are the mouse's biggest environmental impacts [25][26].

The real environmental impact occurs when the same technology is scaled up and mass-produced in a warehouse system [28].

2.3 Sustainability considerations and suggested mitigations

The group's micro-mouse is mostly made of plastic. The key sustainable issue is plastic waste. And this could pose a problem because most plastics are made from non-renewable fossil fuels [27]. During the group's micro-mouse project, we had a lot of useless plastic pieces. If these are not recycled, they end up in a landfill or the natural environment [25].

During the early stages of our project, we could have chosen a more sustainable, eco-friendly plastic for the chassis [27]. We could have designed parts to be reused or replaced rather than thrown away. Specifically, resistors or LEDs that were cut too short. We could have also researched a lighter, more efficient design for the mouse.

2.4 Social impact of similar robot to the micro-mouse

A robot similar to a micro-mouse can have a few positive social impacts, such as carrying out everyday tasks like delivering items, cleaning or transporting goods. This can save workers time, as production speeds up and services become more reliable [23][28]. In hospitals, it can mean patients can obtain their medication more quickly and, ultimately, make daily life more convenient and efficient.

Robots similar to those of a micro-mouse have the potential to create a much safer environment for construction workers entering and mapping unsafe areas or moving loads that are physically demanding for humans [23]. This would drastically reduce the risk of injury or fatigue.

On the other hand, robots built like a micro-mouse, able to reach a specific goal quickly and efficiently, would replace roles that involve routine tasks. Jobs in delivery, cleaning or basic logistics may be reduced [23]. Workers may struggle to find new employment, but retaining their current jobs can increase unemployment and economic inequality.

2.5 Commercial considerations, Intellectual property rights and licensing

There could be a profitable business in the mass production of micro-mouse-type robots capable of executing tasks independently. Working for long periods of time, mapping out dangerous areas where humans can't go. Machines are finding the quickest route to deliver goods to consumers [32]. Companies will only have to pay a small team of engineers to maintain and service these robots.

The company that creates micro-mouse or micro-mouse-type robots must sell their products at a high price to cover the cost of materials and manufacture these robots on a mass scale. If there is a company making these robots, it will need a target market to sell them to. Wherever there is profit, there is also competition, and rival companies will try to create their own version of the same product.

Being first to market can secure intellectual property rights, protect a company's or individual's creation, and make it harder to copy its style or design [29]. If companies violate IPR, they could face a civil lawsuit and a court order granting an injunction.

Licensing is the granting of permission to others to use your design, software, or brand. This also applies to a micro mouse: a person could license their design to a company to manufacture it.

2.6 Legal issues, regulation and compliance for robots like the Micro-Mouse

For a robot like a micro-mouse, the legal side is about ensuring it's safe, compliant with the law, and doesn't harm people. The robot must adhere to certain design regulations to avoid accidents, such as no sharp edges or exposed wiring. Electric components must not overheat or cause shocks [34]. In the UK, we have specific standards that must be met, like the UK Conformity Assessed marking [30]. This ensures the robot won't injure users or cause accidents.

A robot being manufactured and sold must be tested via its power supplies. There could be many issues, such as fire hazards, unsafe charging systems, and environmental harm from disposal [25][34]. Data protection and privacy are also very crucial for a micro-mouse or a robot similar to a micro-mouse. The main issue is that robots may collect data about users or their surroundings. To address situations like this, companies must comply with laws such as the UK GDPR [31].

2.7 Identified risks and suggested mitigations for the micro-mouse robot

The risk could be battery overheating, leaking or catching fire [33]. To avoid this issue, proper insulation and casing can help to prevent overheating. Short circuits or exposed wires could cause damage or injury [33]. To mitigate this effect, insulate all wiring and test the circuits before use.

A micro-mouse is fragile; a collision or a fall could damage components [32]. A bumper could be added to protect the edges. There can also be issues with the software, such as bugs in the code, which could cause incorrect navigation [22]. Fail-safe routines should be used. Small moving parts could cause injury, e.g., fingers caught in wheels. Guards can be installed to reduce this risk.

2.8 Professional codes of conduct, ethics, data security and privacy

Throughout this project, there are guidelines for engineers to act responsibly, such as those from the Institution of Engineering and Technology [29]. This includes ensuring the robot is safe, being honest about its performance, and properly testing and validating the design [29]. Ethically, engineers must not copy code without permission and should be transparent in their work. Data security is also important when using platforms like GitHub, where access must be controlled.

2.9 Health and Safety

Health and safety ensure the micro mouse is safe to build, test, and use. Electrical safety is important to prevent short circuits or shocks [33]. This can be reduced by insulating wires and testing circuits. Motors and components can become hot during use, potentially causing burns [34]. Using heatsinks and allowing cooling helps prevent this. Proper disposal of components helps reduce environmental harm and follows regulations [25].

Section 3: Operating Requirements

3.1 System and Operating Environment

The micro-mouse is an embedded autonomous robot designed to navigate a maze without any prior knowledge of its layout. The system requires a chassis, two stepper motors for movement, infrared and ultrasonic sensors for wall detection, a microcontroller for processing and decision-making, a power supply, wheels, and a steering mechanism. The goal is to travel from the starting position to the target cell in the shortest time possible, with a maximum permitted run time of ten minutes per attempt.

The maze used for this project follows a modified competition standard. While a full IEEE micro-mouse competition uses 16x16 grid of 256 cells, the module requirement specifies an 8x8 grid of 64 cells – one quarter of the standard competition size. Each cell measures 16 cm x 16 cm. The micro-mouse starts in the bottom-left corner of the maze at coordinates (0,0) and must reach the top-right corner at coordinates (7,7), as illustrated in figure below.

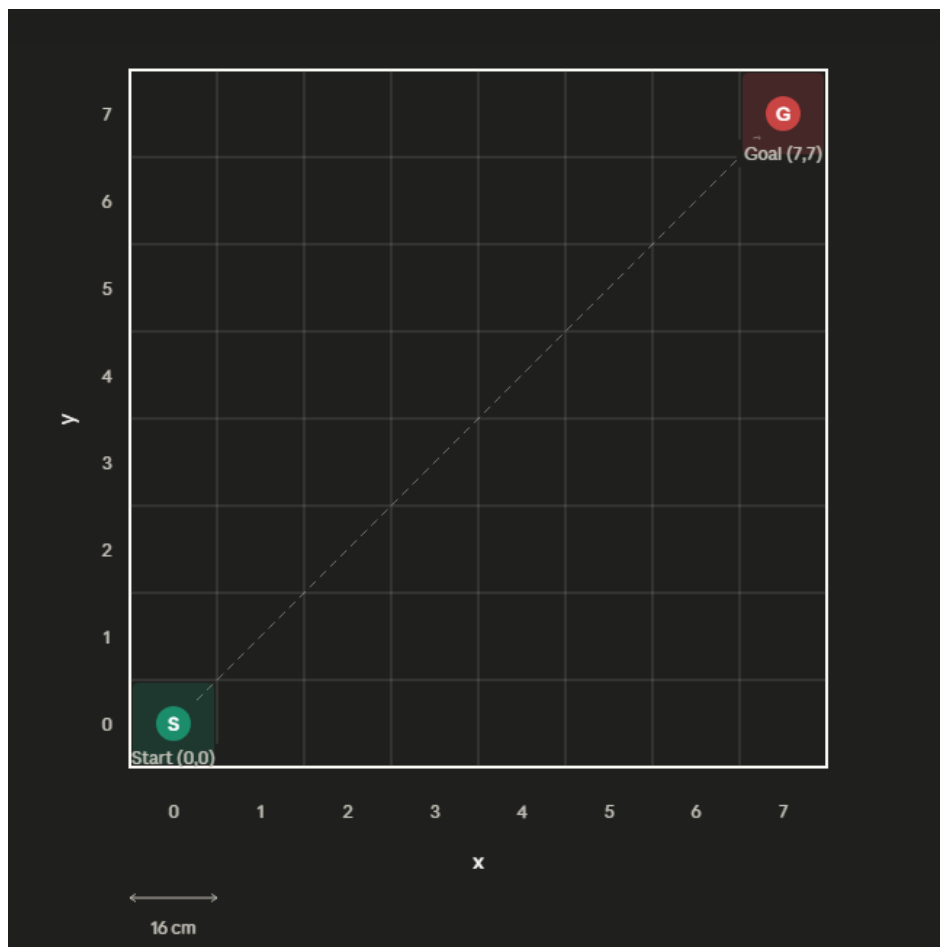


Figure 14: maze example (made on draw.io)

External environmental factors must also be considered during operation. The ambient lighting conditions in the room may affect infrared sensor performance, as strong light sources can introduce noise into sensor readings. The floor surface colour and any surface imperfections may affect wheel traction and movement consistency. Wall dimensions, colour, and thickness are also relevant, as the sensor logic is calibrated around the standard maze wall height of 5 cm. The robot will operate indoors at the university during the final run, which provides a reasonably controlled environment, though these factors remain important during testing.

3.2 Functional Requirements

The micro-mouse must be capable of reading data simultaneously from multiple sensors – infrared sensors on both wings and an ultrasonic sensor at the front and processing this data in real time to determine the presence of walls on the left, right, and directly ahead. All sensor data must be reliable, accurate, and collected at a sufficient rate to support smooth cell-by-cell navigation.

The robot must maintain continuous awareness of its position and orientation within the maze at all times. This is achieved through the flood fill Lee algorithm, which assigns distance values to all cells relative to the goal, dynamically updates these values as new walls are discovered, and always directs the robot towards the lowest-cost accessible neighbouring cell. The position is tracked internally using X and Y coordinates updated after every move, with the orientation tracked as North, East, South, or West.

The micro-mouse must be capable of performing 90-degree left and right turns on the spot, a 180-degree U-turn, and forward movement of exactly one cell at a time. Following a turn, the robot must immediately move forward into the target cell. The system must also correctly identify junction types - dead ends, T-junctions, straight corridors, and open intersections and respond appropriately by updating the flood fill values before selecting the next move.

The software must be developed in C++ using the Keil Studio cloud-based development environment, compiled and uploaded to the OM11043 LPC1768 MBED board.

3.3 Mechanical Requirements

The micro-mouse body, excluding the sensor wings, must fit within a 12cm x 12cm footprint. The sensor wings are permitted to extend over the tops of the maze walls and are not included in this constraint. The overall height of the robot must be a minimum of 8cm, with the sensor wings positioned at approximately 6cm above the ground to maintain reliable line-of-sight detection over the 5cm maze walls.

The robot must leave adequate clearance between its chassis and the maze walls throughout the entire run to prevent collisions during turning and forward movement. Given that each cell is 16cm x 16cm and the robot body is 12cm x 12cm, this leaves approximately 2cm of clearance on each side during straight travel. Turning clearance is the critical constraint and is managed through the differential steering method, where one wheel rotates forward and the other rotates backward, enabling the robot to turn on the spot within the cell boundary.

The estimated total mass of the micro-mouse is approximately 576 g based on component estimates, as detailed in Table 1 of the systems design report. The heavier components including the battery pack and motors are positioned centrally and low on the chassis to maintain a low centre of gravity, improving stability during acceleration, deceleration, and turning. The maximum operational speed for a micro-mouse of this type is estimated at up to 7 m/s, though in practice the step count and timing will be calibrated on hardware to achieve accurate and repeatable cell-to-cell movement.

The broader concerns discussed in Section 2 also informed these requirements. Health and safety considerations (Section 2.9) required that the robot operate within its maze boundaries at all times, which informed the clearance requirements and the need for reliable sensor-based wall detection. Sustainability objectives (Section 2.3) influenced the choice of modular, maintainable mechanical design, ensuring individual components can be replaced or repaired without discarding the entire chassis.

Section 4: Specification and Costs of a Micro-Mouse Robot

4.1 Component Specifications and Justification

The component selection for the micro-mouse was driven by the functional and mechanical requirements established in Section 3. Each component was chosen to meet the specific needs of an 8x8 maze-solving robot while remaining within the constraints of the university-provided hardware kit.

The OM11043 LPC1768 MBED board serves as the central microcontroller. It is based on a 32-bit ARM Cortex-M3 processor running at 96MHz with 512KB of flash memory and 32KB RAM. This is significantly more capable than 8-bit alternatives, and its comprehensive peripheral support – including SPI, I2C, UART, CAN, GPIO, ADC, and PWM – provides more than sufficient interfacing capability for the stepper motors, IR sensors, and the ultrasonic sensor used in this project. The board also supports Keil Studio for C++ development requirement.

Two Y129 unipolar stepper motors provide the drive system. Stepper motors were selected over DC motors due to their superior positional accuracy, which is critical for making precise 90-degree and 180-degree turns within the constrained 16cm x 16cm cell dimensions. Each motor operates at 12V and draws 160mA per coil, which is within the capability of the power distribution circuit described in section 5.

The L297 stepper motor controller chip handles the phase sequencing for each motor, accepting simple step and direction signals from the MBED and translating them into the four-phase drive sequences required by the unipolar stepper motors. This offloads the sequencing logic from the microcontroller and simplifies the firmware implementation. The ULN2003 motor driver handles the high-current sinking required by the stepper coils, with its internal flyback diodes providing protection against voltage spikes – an important consideration given the inductive nature of the motor coils.

Six OP165D infrared emitters and six IS471 infrared receivers are used across the two sensor wings, with three emitter-receiver pairs per wing. These detect wall presence on the left and right sides of the robot. The IS471 receiver includes an integrated demodulator, making it resistant to ambient light interference and well suited to variable indoor lighting conditions. An HC-SR04 ultrasonic sensor is mounted at the front of the chassis for forward wall detection, with a detection range of 2cm to 400cm and a 15-degree cone angle, providing reliable detection of walls within the maze cell spacing.

The LM7805 linear voltage regulator steps the 12V battery supply down to a stable 5V rail for all logic-level components, including the MBED, L297, and sensor circuitry. This separation of the high-voltage motor rail from the 5V logic rail ensures sensitive components are protected from voltage fluctuations caused by motor switching.

Two ceramic decoupling capacitors are used in conjunction with LM7805 regulator – a 0.33uF capacitor on the output side, as recommended by the LM7805 datasheet. These suppress voltage fluctuations and prevent the regulator from oscillating under varying load conditions, ensuring a clean and stable 5V supply to the MBED, L297, and sensor circuitry.

The system is powered by eight AA NIMH rechargeable batteries in series, providing 9.6V (or up to 12V with alkaline cells). NIMH chemistry was selected for its improved

safety profile over lithium-based alternatives– the batteries are less prone to swelling or combustion under overcharge conditions and for its recyclability, supporting the sustainability objectives discussed in section 2.3.

4.2 Budget and Costing

All components were provided by the university for this project, except for the batteries which were purchased separately. The table below shows the estimated cost if each component were purchased independently. Prices are sourced from The Pi Hut and RS Components as the reference suppliers.

Component	Qty	Unit Price (Inc. VAT)	Total Price	Unit Weight	Total Weight	Supplier
MBED LPC1768	1	£36.31	£36.31	30g	30g	ARM EMBED
L297 Stepper Motor Controller	2	£12.00	£24.00	3g	6g	The Pi Hut
Y129 Unipolar Stepper Motor	2	£12.00	£24.00	220g	440g	The Pi Hut
ULN2003 Motor Driver	2	£0.86	£1.72	12g	24g	RS Components
LM7805 Voltage Regulator	1	£0.80	£0.80	5g	5g	RS Components
OP165D IR Emitter	6	£1.00	£6.00	1g	6g	The Pi Hut
IS471 IR Receiver	6	£0.80	£4.80	1g	6g	The Pi Hut
HC-SR04 Ultrasonic Sensor	1	£3.00	£3.00	9g	9g	The Pi Hut
8x AA Battery Holder (with switch)	1	£5.20	£5.20	75g	75g	The Pi Hut
AA Alkaline Batteries	8	£1.13(per cell)	£9.00	23g	184g	The Pi Hut/Supermarket
Chassis	1	N/A	N/A	50g	50g	University Provided

Ceramic Decoupling Capacitors uF 0.33	1	£0.10	£0.10	<1g	<1g	RS Components
Ceramic Decoupling Capacitors uF 0.10	1	£0.10	£0.10	<1g	<1g	RS Components
Total		£73.30	£115.03	429g	835g	

4.3 Sustainability and Environmental Considerations

The component selection reflects several sustainability objectives identified in section 2. The use of NiMH batteries instead of lithium-ion or nickel-cadmium alternatives reduces environmental impact – NiMH cells contain no highly toxic heavy metals and are recyclable at standard electronic waste facilities, aligning with the Waste Electrical and Electronics Equipment Directive discussed in Section 2.2.

The modular Vero-board construction described in Section 7 also contributes to sustainability: individual boards can be replaced if damaged rather than discarding the entire system, and expensive integrated circuits such as the MBED, L297, and ULN2003 are mounted in sockets rather than soldered directly, allowing them to be removed and reused in future projects. Where possible, components were chosen for longevity and repairability rather than single-use convenience, supporting the circular economy principles outlined in section 2.3.

4.4 Regulatory Compliance

The electronic components used in this project are standard through-hole components operating within their rated voltage and current limits, as detailed in Tables 2 and 3 of the systems design report. The system does not transmit wirelessly and therefore does not fall under radio frequency emissions regulations. The battery selection and motor driver circuitry include appropriate protection measures - flyback diodes on the motor drivers and a regulated 5V supply for logic components - ensuring the design does not pose an electrical safety risk under normal operation. All components used are RoHS compliant where applicable, in line with the requirements of the Restriction of Hazardous Substances Directive.

Section 5: Hardware Design & Mechanics

5.1 Chassis Dimensions, Layout & Design

The heavy components, such as the batteries and motors, are positioned centrally and symmetrically to maintain a low centre of gravity. IR sensors are mounted on different 'wing' boards to hover over the walls, enabling precise wall detection. Ultrasonic sensor mounted at the very front of the chassis to ensure early front-wall detection for the micro mouse deceleration. The MBED was mounted on the back of the chassis with protection in case of a wall crash. Each system was divided into different modules on their own Veroboard's.

5.2 Power Distribution Circuits

Power distribution is managed via a primary +12V rail that feeds the high-current demand motor subsystems, and the rest of the components requiring +5V through a voltage regulator, having a 7V drop. By separating the motors from the rest of the circuitry we ensure the rest of the components are not damaged from an excessive current load.

5.3 Motor Driver Circuits & Motors

The motor system uses two unipolar stepper motors which require +12V, driven by a two-stage circuit that consist of an L297 stepper motor controller and a ULN2003 motor driver chip (Power Darlington transistors).

- The L297 acts as a sequence generator, oversees the logic generation for the stepper motors. It takes simple step and direction commands from the MBED and translates them into 4-phase sequences. The system operates as a voltage-driven step generator.
- The ULN2003 chip handles the high-current sinking required by the stepper coils. Connecting the COM pin of the ULN2003 chip directly to +12V activates the chip's internal flyback diodes, providing complete protection against voltage spikes from motors. An additional protection was placed with external diodes between the motors and the ULN2003 chips.

5.4 Sensor Configuration and Sensor Circuits

The micro mouse uses infrared (IR) sensor pairs consisting of emitters and receivers, mounted on the front edges of the chassis as wings to detect the walls from above; each wing with 3 IR sensors. To avoid mutual interference for the middle sensor which

should be continuously detecting the wall we placed it facing the opposite way of the other sensors, and further shielding provided to each sensor avoiding mutual and external interference. To prevent electrical interference from the motors from corrupting the delicate signal of the sensor readings, the sensor connections are isolated from the high-current connections.

The output sense pins of the IR receivers are directly routed to the GPIO pins of the MBED, in this case pins 6-11. For front wall detection an ultrasonic sensor was mounted at the front of the chassis. The only concern was the output of the ultrasonic sensor being +5V, so to overcome the issue a voltage divider circuit was implemented. Two of the pins are connected to the MBED, Trigger (output) and Echo (input). The MBED sends a signal to the trigger pin and uses internal hardware timers to measure the returning echo pulse measuring the distance forward.

5.5 Micro-Controller Board Circuitry

The brain of the mouse in this case is the MBED LPC1768. It is mounted using female headers ensuring the pins are not soldered directly to the Vero-board. The MBED acts as a bridge between the sensor inputs and the motor driver outputs. The unused pins of the MBED were left floating to conserve connection space and minimise unnecessary wiring, further contributing to noise reduction.

5.6 Power Source

The system is powered by a 12V battery pack consisting of x8 AA NiMH batteries, which create the primary voltage rail for the motor coils. The choice of NiMH batteries was a safety consideration as well. NiMH batteries are more chemically stable compared to batteries like Li batteries. They provide safety and performance. Since they are more stable, they are less prone to combustion or swelling if overcharged, or excessive heat. The battery pack is securely strapped to the chassis to prevent shifting during acceleration and deceleration. They are placed at the bottom of the chassis near the centre of gravity to generate a greater balance for the micro mouse.

Our project strongly aligns with electronic sustainability objectives using NiMH batteries. Unlike other batteries like NiCd or Lead acid batteries, NiMH batteries are entirely free of highly toxic heavy metals, making them significantly safer for the environment and the user. Upon further disposal of the batteries having NiMH cells are more environmentally friendly and recyclable at electronic waste facilities.

Section 6: Software Design & Logic

6.1 Overall System Discussion

The micro-mouse software is written in C++ using the Keil Studio cloud-based development environment and compiled for the MBED LPC1768 using the MBED OS 6 framework. The software is structured as a single continuous control loop that repeats until the goal cell is reached. The code itself has been sectioned off for many different functions looping into each other. The robot reads its sensors, update internal wall mapping, selects the best direction using the flood fill Lee algorithm, executes the appropriate motor movement, updates its position, and reruns the flood fill calculation before repeating the cycle.

The overall design follows a sense plan act model. The robot first senses its environment through its sensor suite, plans its next move using the flood fill Lee numbers, and then acts by commanding the stepper motors. This cycle continues cell by cell until the target coordinate [7][7] is reached, at which point the program exits the loop and stops. The flood fill values are recalculated after every move to account for any newly discovered walls, ensuring the robot always takes the shortest available path to the goal given its current knowledge of the maze.

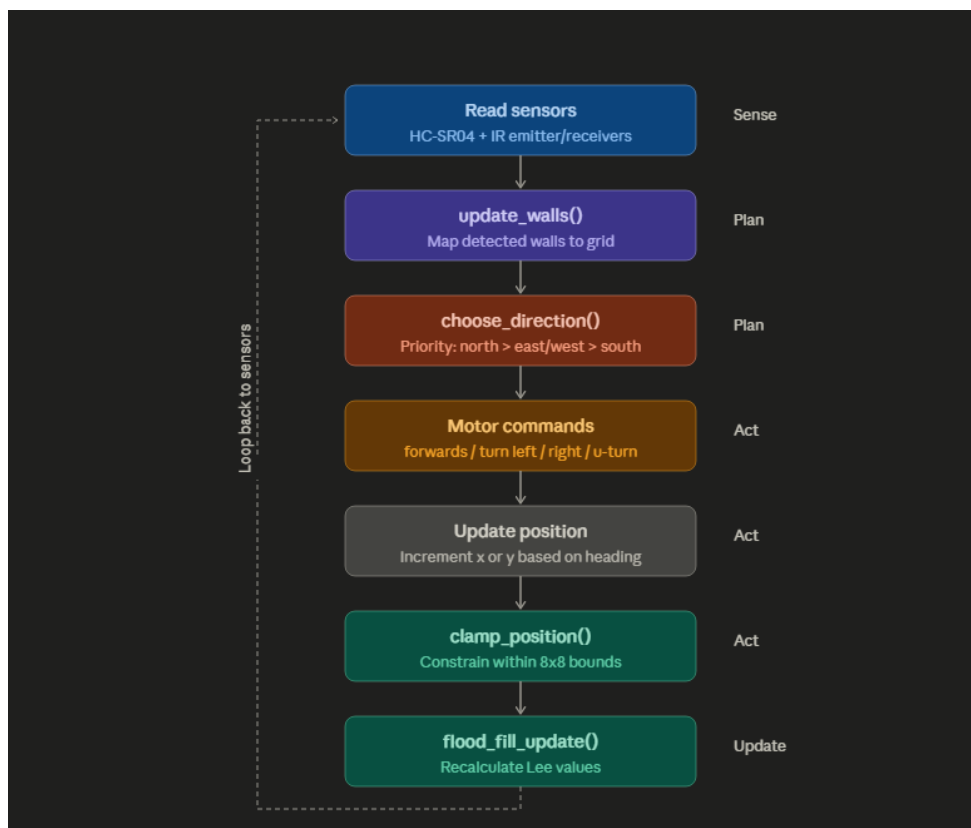


Figure 15: Block diagram of code priority

6.2 Software Structure

The software is divided into the following functional groups, each implemented as one or more C++ functions:

Sensor reading is handled by `check_us_distance()`, `check_left_sensors()`, and `check_right_sensors()`. These are wrapped into three Boolean helper functions `wall_in_front()`, `wall_to_left()`, and `wall_to_right()` which returns true or false depending on whether a wall is detected on each side.

Wall mapping is handled by `update_walls ()` and `share_walls ()`. The former translates three Boolean sensor readings into absolute compass directions based on the robot's current orientation and records them into the `number_of_maze_walls` array. The latter propagates each newly discovered wall to the neighbouring cell on the other side, keeping the map consistent.

Flood fill is handled by `flood_fill_setup()` and `flood_fill_update()`. The setup function runs once at startup, initialises all cell values, pre-loads the outer perimeter walls, and performs an initial BFS from the goal cell. The update function reruns the BFS every time the robot moves, incorporating any newly discovered walls.

Navigate is handled by `choose_direction()`, which checks the flood fill Lee numbers of all accessible neighbours and returns the direction with the lowest value. The main loop then calculates the number of turns required and commands the motors accordingly. Motor control is handled by `steps ()`, `forwards ()`, `turn left ()`, `turn right ()`, and `u_turn()`. These generate the step and direction pulses for the two stepper motors. Safety is handled by `clamp position ()`, which prevents the tracked X/Y coordinates from exceeding the valid 0-7 range in the event of a sensor error.

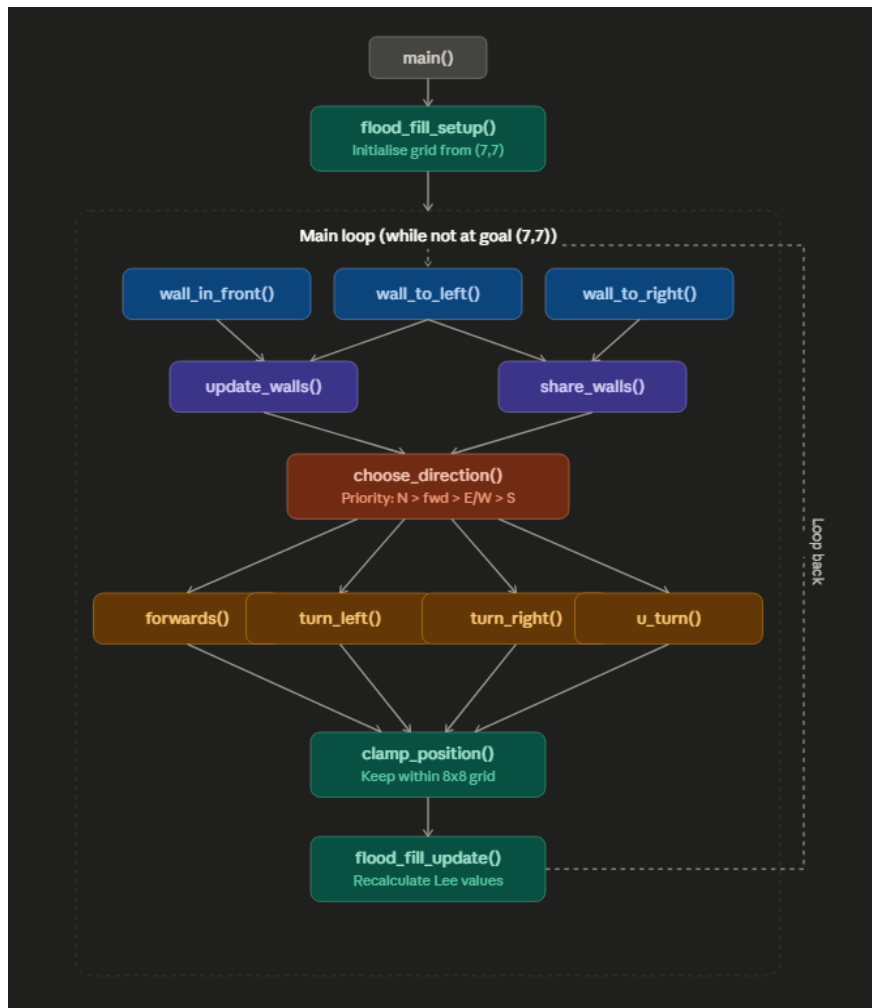


Figure 16: Block diagram of function call hierarchy (made on draw.io)

6.3 Sensor Logic used for Navigation

Three sensor systems are used to detect walls. The ultrasonic HC-SR04 sensor detects walls directly in front of the robot. A 10-microsecond trigger pulse is sent to the sensor, and the time taken for the echo to return is measured using the MBED hardware timer. The distance is calculated using the formula:

```
//Speed of sound = 0.343 mm/us
us_distance_mm = (echo_time*343)/2000;
```

Figure 17: Code for calculating distance

Where echo time is measured in microseconds and 343 mm/us is the speed of sound. A wall is considered present if the measured distance is less than 80mm, which corresponds to approximately half a cell width and provides reliable detection before the robot reaches the wall.

The left and right walls are detected using two banks of three infrared emitter receiver pairs mounted on the sensor wings. Each bank is connected to a 3-bit BusIn and read as a combined value. A reading of 7 (binary 111, all three sensors triggered) indicates a wall is present on that side:

```
int check_left_sensors(){
    int left_wall_value = Left_Sensors.read();
    if (left_wall_value == 7){
        return 1;
    }
    // can add elseif for alignment issues but need hardware to test
    else {
        return 0;
    }
}
```

Figure 18: Left wall check

These three Boolean results – front, left, right – are read at the start of every loop iteration and passed directly into update_wall().

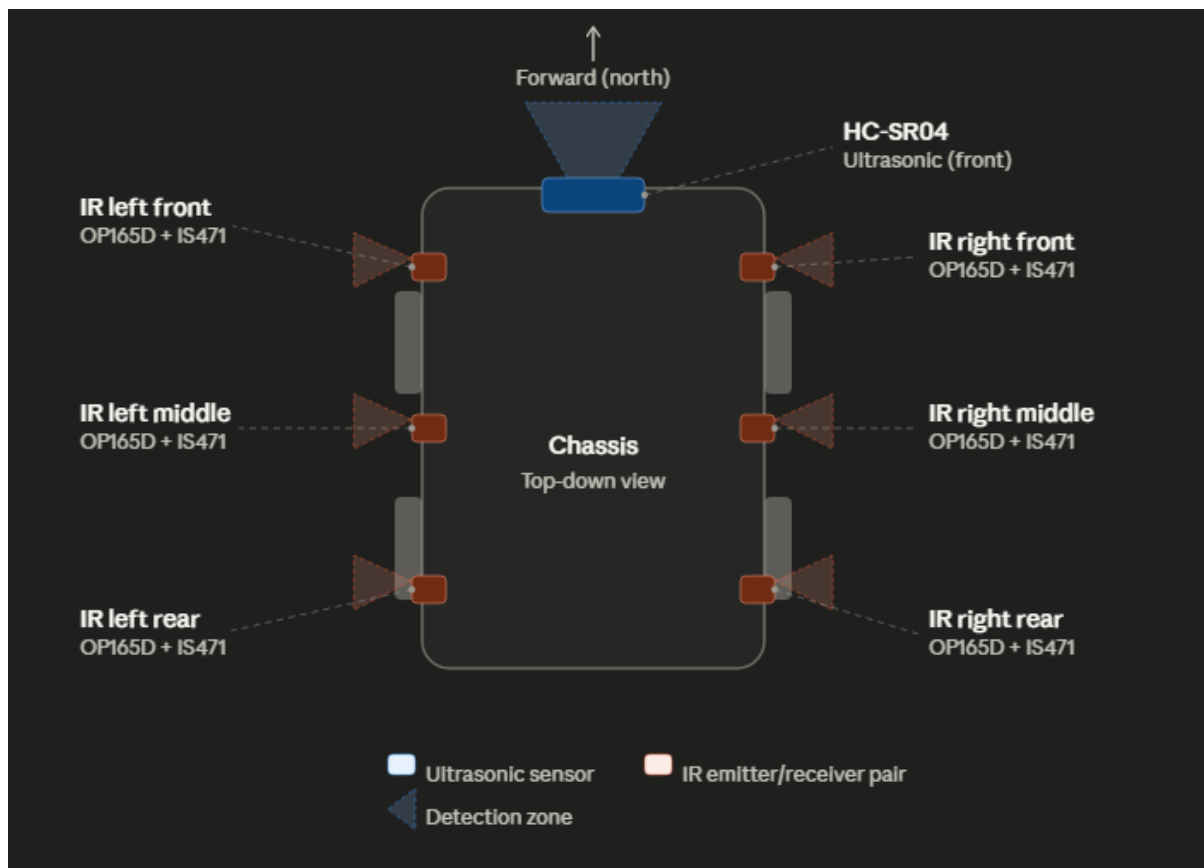


Figure 19: Diagram of sensor locations (made on draw.io)

6.4 Post Processing of Sensory Information

Raw sensor readings are translated into absolute wall directions by `update_walls()`, which uses the robot's current orientation to map relative sensor positions – front, left, right – to absolute compass directions: North, East, South, West. Walls are stored in the `number_of_maze_walls` array using a 4-bit bitmask per cell, where bit 0 represents North (value 1), bit 1 represents East (value 2), bit 2 represents South (value 4), and bit 3 represents West (value 8).

For example, when the robot is facing North (`orientation == 0`), a wall detected in front maps to North, a wall detected to the left maps to West, and a wall detected to the right maps to East:

```
if (front) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 1; // Mark North Wall, Front is North (1)
if (left)  number_of_maze_walls[mouse_position_x][mouse_position_y] |= 8; // Mark West Wall, Left is West (8)
if (right) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 2; // Mark East Wall, Right is East (2)
```

Figure 20: Stores wall information

This mapping rotates correctly for all four orientations. For example, when facing East (`orientation == 1`), the front sensor maps to East, the left sensor maps to North, and the right sensor maps to South.

Once a wall is marked on the current cell, `share_walls()` is called immediately to propagate it to the neighbouring cell on the side. For example, a North wall on cell `[x][y]` becomes a south wall on cell `[x][y+1]`:

```
if((number_of_maze_walls[x][y] & 1) && y < 7)
    number_of_maze_walls[x][y+1] |= 4;
```

Figure 21: Share wall data with neighbour

This ensures the wall map is always consistent from both sides of every wall, which is essential for the flood fill to correctly rout around discovered walls.

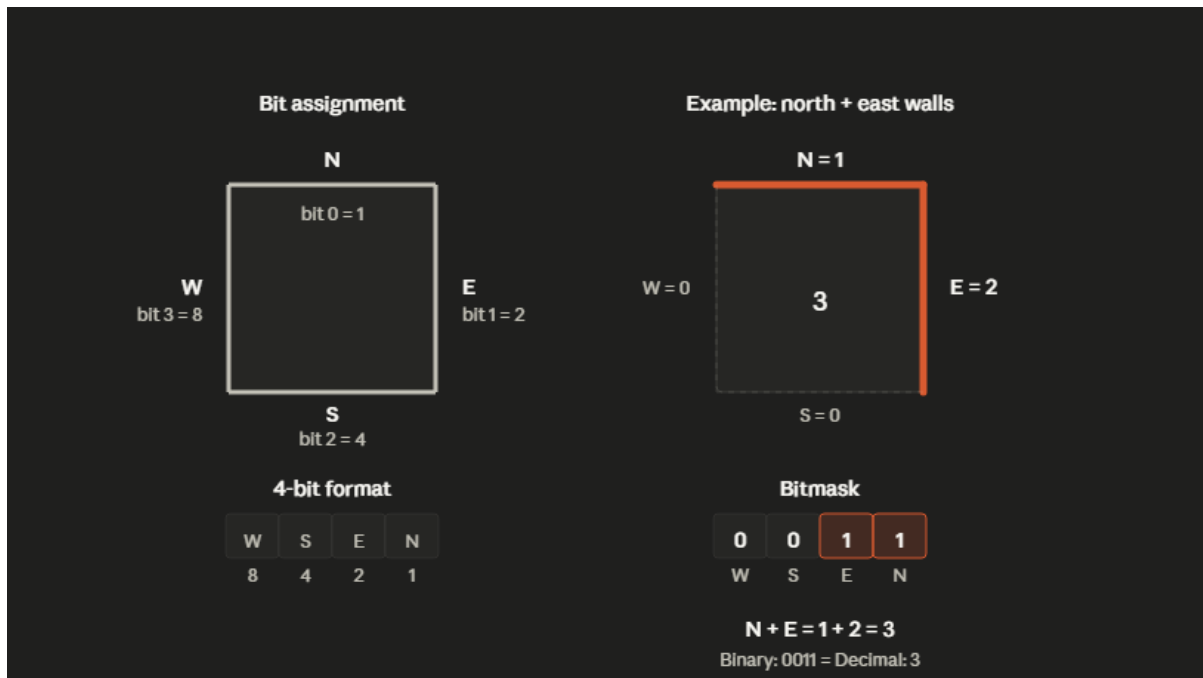


Figure 22: Example of bitmask

6.5 State Machine & Navigation Implementation

The main navigation loop operates as a continuous cycle. At the start of each iteration the sensor readings are taken, and `update_walls()` is called to record any detected walls. The program then checks where the goal has been reached. If the robot's position is [7][7] the loop breaks and the program ends. Otherwise `choose_direction()` is called to select the best cell.

`Choose_direction()` checks all four neighbours of the current cell. Any neighbour blocked by a wall in the `number_of_maze_walls` array is skipped. Of the remaining accessible neighbours, it returns the direction of the one with the lowest Lee number:

```
//choose best direction using flood fill Lee numbers
int choose_direction(){
    int best_val = 9999;
    int best_dir = -1; // N=0, E=1, S=2, W=3

    // Check North - no north wall and in bounds
    if(mouse_position_y < 7 && !(number_of_maze_walls[mouse_position_x][mouse_position_y] & 1)){
        if(number_of_maze_cells[mouse_position_x][mouse_position_y+1] < best_val){
            best_val = number_of_maze_cells[mouse_position_x][mouse_position_y+1];
            best_dir = 0;
        }
    }
}
```

Figure 23: Finds best direction to move using Lee numbers

The number of turns required to face the target direction is calculated using modular arithmetic:

```
// Work out how many right turns needed to face target direction  
int turns_needed = (target_dir - orientation + 4) % 4;
```

Figure 24: Code to calculates necessary amount of turns

This produces 0 for straight ahead, 1 for a single right turn, 2 for a U-turn, and 3 for a single left turn. The appropriate motor function is called, followed by forwards () in every case where a turn was needed. After moving, the orientation and position are updated directly in main:

```
// Update orientation and position after moving  
orientation = target_dir;  
if(orientation == 0) mouse_position_y++;  
else if(orientation == 1) mouse_position_x++;  
else if(orientation == 2) mouse_position_y--;  
else if(orientation == 3) mouse_position_x--;
```

Figure 25: Code to update orientation

Clamp_postion() is then called to snap coordinated back within the valid 0-7 range if a sensor error has produced an out-of-bounds value. Finally, flood_fill_update() reruns the BFS to update all Lee numbers before the next iteration begins.



Figure 26: Flowchart of order of process

6.6 Mapping and Path Analysis Solutions

The flood fill Lee algorithm is implemented in two stages. At startup, `flood_fill_setup()` performs the following steps in order: it initialises all cells in `number_of_maze_cells` to zero, calls `setup_perimeter_walls()` to pre-load the outer boundary walls, and then runs a breadth-first search (BFS) from the goal cell `[7][7]` outward, assigning each cell an incrementing distance value with no internal wall knowledge:

```
number_of_maze_cells[7][7] = 0;
visited[7][7] = true;
queue_x[tail] = 7; queue_y[tail] = 7; tail++;

// Process till empty
while(head != tail){
    int cx = queue_x[head];
    int cy = queue_y[head];
    head++;

    int next_val = number_of_maze_cells[cx][cy] + 1;

    // Check North neighbour, if not visited assign Lee number and add to queue
    if(cy < 7 && !visited[cx][cy+1]){
        number_of_maze_cells[cx][cy+1] = next_val;
```

Figure 27: Code for flood fill

This produces an initial distance map where every cell has a Lee number representing its Manhattan distance from the goal, assuming open corridors throughout.

After each move, `flood_fill_update()` resets all cells to 999 and returns the BFS from `[7][7]`, this time skipping any boundary that has a discovered wall:

```
if(cy < 7 && !(number_of_maze_walls[cx][cy] & 1) && number_of_maze_cells[cx][cy+1] == 999){
    number_of_maze_cells[cx][cy+1] = next_val;
```

Figure 28: Checks for walls starting North

This means the Lee numbers always reflect the shortest available path to the goal given the walls discovered so far. As more walls are found during exploration, the numbers dynamically reroute the robot away from dead ends and toward the true shortest accessible path. The use of 999 as the reset value and the wall check together ensure that cells isolated by walls remain unreachable and are never chosen as a destination.

The outer perimeter walls are pre-loaded by `setup_perimeter_walls()` before the first BFS runs, marking South walls on the entire bottom row, North walls on the entire top row, West walls on the entire left column, and East walls on the entire right column. This prevents the flood fill from ever routing outside the 8x8 grid boundary, removing any need for additional boundary checks inside the BFS loops.

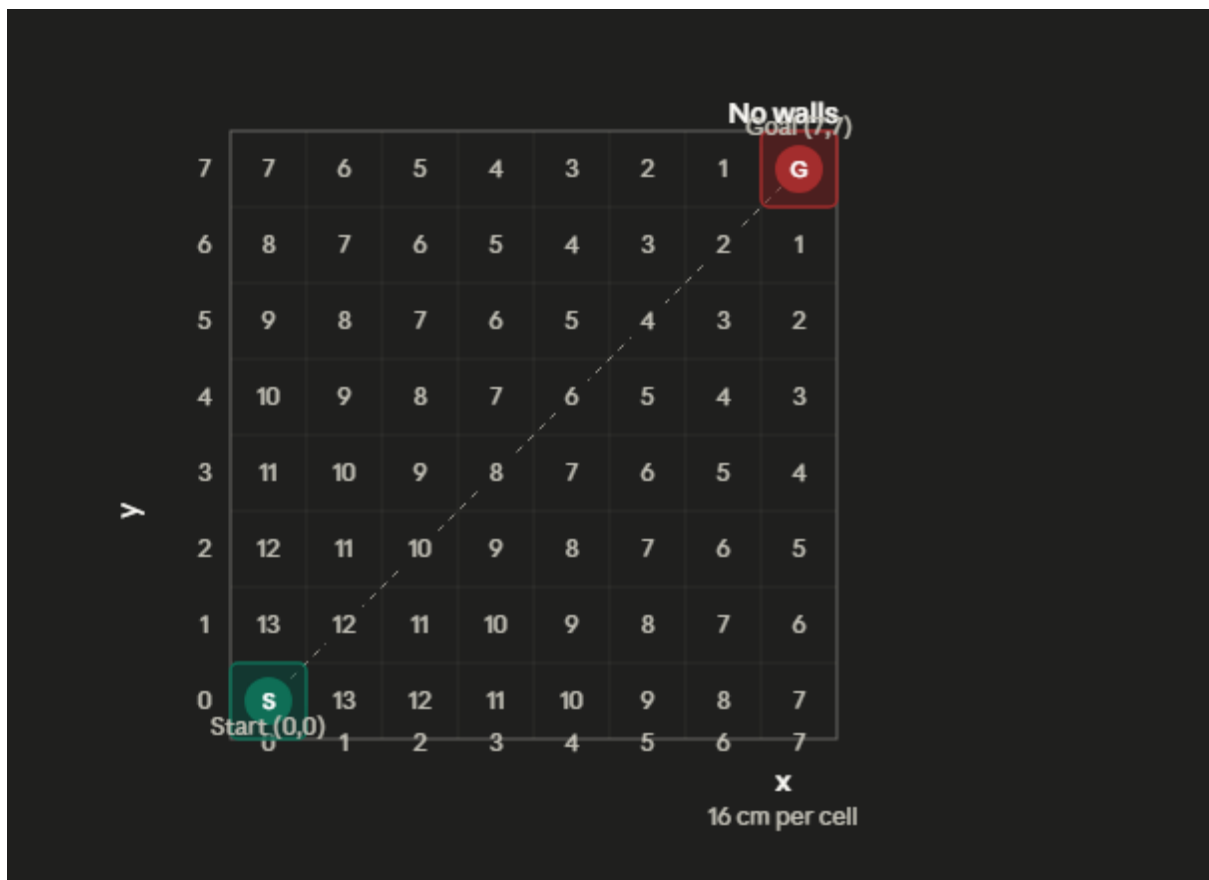


Figure 29: Maze without walls (made on draw.io)

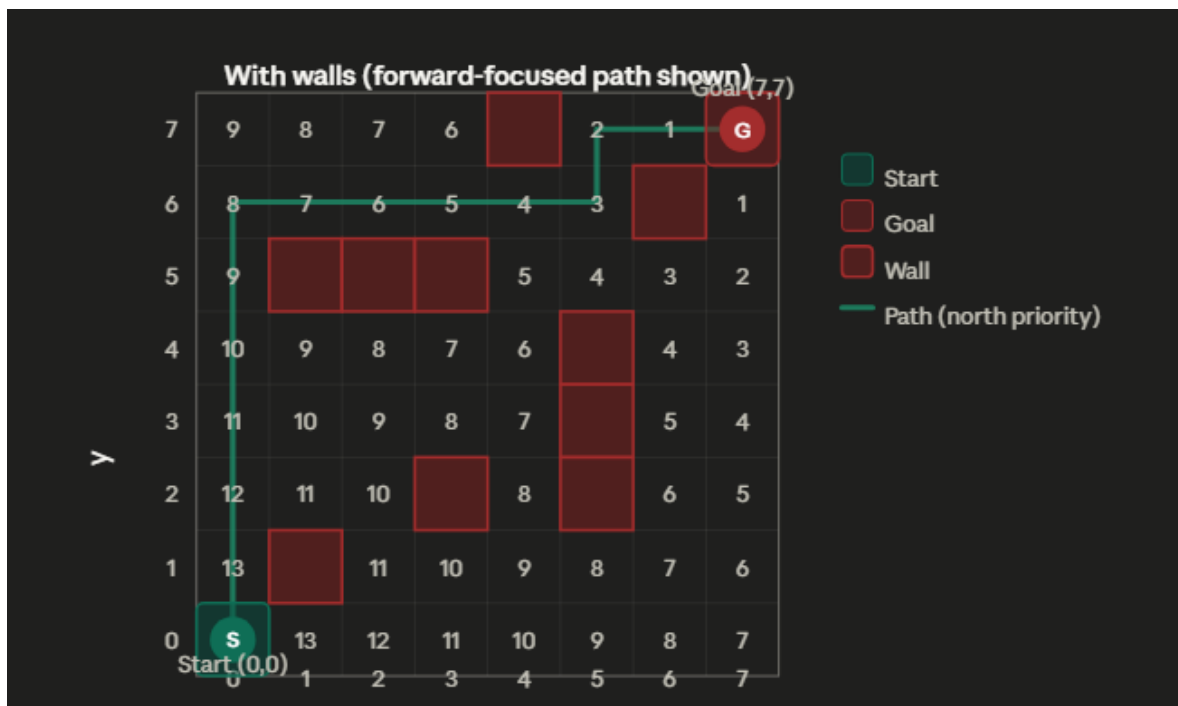


Figure 30: Maze with walls example (made on draw.io)

6.7 Power Management and Efficiencies

The software contributes to power efficiency primarily through the flood fill algorithm itself. By always selecting the cell with the lowest Lee number, the robot takes the most direct available route to the goal rather than wandering the maze randomly. This minimises the total distance driven and therefore the total energy consumed by the stepper motors, which are the highest-current consumers in the system at 160mA per coil.

Two ceramic decoupling capacitors – 0.33uF on the input and 0.1uF on the output of the LM7805 voltage regulator – work in conjunction with the software timing to maintain a clean 5V supply rail. The `wait_us()` calls place between sensor reads and motor commands provide a settling period that allows transient voltage fluctuations caused by motor switching to dissipate before the next sensor reading is taken. Without this settling time, motor switching noise could corrupt sensor readings and cause unnecessary correction movements, wasting both time and energy.

The ultrasonic sensor is only triggered at the start of each loop iteration rather than continuously, avoiding unnecessary power draw between readings. The sensor is not active during motor movement, which also reduces the risk of electrical interference from motor switching affecting the echo measurement.

As noted in Section 4, the NiMH battery chemistry supports stable power delivery throughout a run. NiMH cells maintain a relatively flat discharge curve across most of their capacity, providing consistent voltage to the motor driver circuits and reducing the risk of undervoltage failures mid-run that could cause the robot to lose positional accuracy or stall.

Section 7: Implementation of Hardware

The system was divided into separate functional modules, each built on its own independent Veroboard. This independent module built provided significant advantages in mounting flexibility, independent debugging, and separation of electrical noise.

- The MBED LPC1768 was mounted on its own Veroboard making it the central system. By having the MBED isolated the pins are physically distanced from the high electrical noise of the motors, EMI noise.
- Each motor, along its associated driver chips and ULN2003, was built onto their own Veroboard's. This allowed the motor drivers to be mounted near the physical motors, minimising length of high-current wires which can cause power loss if too long.
- The ultrasonic sensor was mounted on a separate, small Veroboard in the front of the micro mouse. Allowing the sensor to be placed at the front of the mouse gives accurate forward collision detection.
- Left and right IR sensors were soldered onto separate 'wing' boards. By having the sensors on the wings separately allows precise tuning. They are mounted in this way to optimise the detection of the walls from above. Angling the sensors also allowed us to overcome the Veroboard's geometry.

Since the circuitry is distributed across multiple boards, each board had to be bridged through a complex wiring system to the common power rail and ground. The sensors and the +5V logic components were bridged to a 'main' board, to the output of the voltage regulator to be powered. The step/direction of the driver chips and sensor outputs were bridged to the central MBED board through standard wires.

By having a distributed Veroboard implementation electronic sustainability is also addressed. With this modular approach, only if a specific component is damaged and the Vero-board is affected, only that specific small Vero-board needs replacement instead of a whole Vero-board full of components, which can also be damaged causing waste. The expensive integrated components like the MBED, L297, and ULN2003 chips were mounted using sockets. This allows for the chips to be reused in further projects. Scrap Vero-board cuts were also used optimising recycle of material usage.

Section 8: Testing

8.1 Design for Test Approaches Adopted

Effective testing of both software states and physical hardware behaviour relied on the micro-mouse being designed with observability in mind from the outset. Four main approaches were used across the development cycle: serial/terminal print debugging, LED state indicators, oscilloscope probing of motor driver pins and software simulation in Keil Studio.

Serial terminal debugging via the MBED serial output was the primary mechanism for exposing internal software state. The key variables were `mouse_position_x`, `mouse_position_y`, `orientatation`, the `number_of_maze_cells` array and the `number_of_maze_walls` array. These were printed to the Keil studio terminal at the start of every main loop iteration. The team could trace the robot's decision-making process step by step without requiring the physical robot to be moving enabling logical errors in `choose_direction()`, `update_walls()` and `sahre_walls` to be identified and corrected early. Each function operates on shared global arrays, printing array contents after individual function calls made it straightforward to verify that each function produced the correct output for a given input before integration.

LED indicators on the MBED board were used to provide a fast visual signal of discrete internal states during hardware bench testing where attaching serial terminal is not always practical. An LED was toggled to indicate when a wall had been detected on each side, providing immediate visual feedback the sensor pipeline was triggering correctly without needing to read terminal output. This proved particularly useful during initial sensor wiring checks where a simple on/off signal was sufficient to confirm the IS471 receivers and HC-SR04 were returning live data to the MBED GPIO pins.

An oscilloscope was used to probe the step and direction output pins of the MBED connected to the L297 stepper motor controller inputs and the phase output pins of the ULN2003 motor driver. This allowed the team to verify that the 500 Hz step pulse frequency generated by the `steps()` function with `wait_us(1000)` for both the high and low phase was being produced correctly and the ULN2003 was switching the four motor coil phases in the correct unipolar sequence for both forward and reverse rotation. The oscilloscope also confirmed the no significant voltage spikes appeared on the 5V logic rail during motor switching, validating the decoupling capacitor and flyback diode protection design.

Keil Studio simulation was used throughout software development to verify navigation logic before any hardware was connected. By running the code in the simulator and supplying fixed sensor return values, the flood fill setup, wall mapping and navigation

state machine could all be validated in a controlled environment with repeatable inputs. This caught the missing `update_walls()` function call and the incorrect `u_turn()` step count before either defect could cause unpredictable behaviour on the physical robot.

At the hardware level, the modular Veroboard construction described in Section 7 directly supported all four approaches by allowing each sub-circuit to be tested in isolation before connection to the rest of the system. Faults could be attributed to a specific module rather than requiring the full robot to be assembled and operational before testing could begin.

8.2 Testing Methodology

The testing strategy followed a structured bottom-up approach, beginning with individual components and progressing through software unit testing, hardware integration testing and finally system level verification. This methodology was chosen because it isolates failures at the lowest possible level preventing compounding errors from obscuring the root cause of a fault and ensuring each layer of the system is confirmed correct before the next layer depends on it.

At the component level, White Box testing was applied to each hardware sub circuit and each software function in isolation. Sensor circuits were bench tested with known physical input conditions, a surface place at a measurement distance or no object present and raw output values were observed via the serial terminal and LED indicators. Motor driver circuits were tested by issuing direct GPIO step and direction pulses and confirming correct phase sequencing on the oscilloscope, independent of the navigation software.

At the software level, white box unit testing was applied to each function individually. Functions were called with known inputs, and the resulting state of global arrays was inspected via the serial terminal. This confirmed the correctness of `update_walls()`, `share_walls()`, `flood_fill_setup()`, `flood_fill_update()` and `chosen_direction()` independently before they were composed into the main navigation loop.

Integration testing then verified the interaction between software functions and hardware peripherals. Sensor readings were fed into `update_walls()` under controlled conditions and the resulting wall array contents were printed to confirm that the relative to absolute direction mapping was correct for all four orientations. Motor control functions were called from the main loop, and the physical robot response was observed alongside oscilloscope traces to verify that step counts and timing produced the correct movement.

Finally, system level Black box testing was applied to the complete navigation state machine using the defined test vectors in section 8.5 and 8.6. These tests specified expected behaviour based solely on sensor input combinations without reference to internal state, verifying that the robot's observable outputs matched the functional requirements. System level testing was conducted primarily through the Keil Studio simulator and serial terminal, as full physical maze runs had not been completed at the time of writing this section.

8.3 Testing of Sensor Circuits & Outcomes

Sensor testing was conducted at two levels: both level hardware and software level integration testing via the serial terminal. Each sensor was connected to the MBED in isolation and raw output values were observed before being put into the full wall detection logic.

The IS471 infrared receiver modules on each wing were tested by presenting a flat surface at the expected wall detection distance of approximately 5cm corresponding to the top of the standard maze wall and reading the BusIn value on the terminal. With a wall present, all three receivers on each wing triggered simultaneously returning a BusIn value of 7 9(binary 111), which is the threshold used by `check_left_sensors()` and `check_right_sensors()` to flag a wall. With no wall present, BusIn value returns 0. the integrated de-modular in the IS471 was verified to reject ambient fluorescent lighting with no false positives were observed when the sensors were exposed to typical laboratory overhead lighting without a wall present.

The HC-SR04 ultrasonic sensor was tested by presenting a flat vertical surface at measured distances and comparing the value returned by `check_us_distance()` against a ruler measurement. At 60mm, the sensor returned a consistent reading within approximately 5mm of the true distance, well within the 80mm threshold used for `wall_in_front()`. At 200mm with no wall present, the returned value exceeded 80mm, correctly returning false from `wall_in_front()`. The voltage divider circuit used to step the 5V echo output down to a safe level for the MBED GPIO input pin was verified to produce a stable logic-level signal without degradation.

Sensor tests are shown below:

Test	Sensor	Condition	Expected output	Our Outcome	Pass/fail
Test 1	Left wing	5cm from wall	Wall is read (binary 111)	Busin return 7, wall to left true	Pass
Test 2	Left wing	No object within	No wall to read	Busin return 0, wall to left false	Pass

		sensor range			
Test 3	Right wing	5cm from wall	Wall is read (binary 111)	Busin return 7, wall to right true	Pass
Test 4	Right wing	No object within sensor range	No wall to read	Busin return 0, wall to right false	Pass
Test 5	Ultrasonic	Flat surface 60mm in front	Distance <80, wall is read	Distance is read, wall is flagged as true	Pass
Test 6	Ultrasonic	No obstruction for 200mm	Distance >80, no wall	No wall to flag, return false	Pass

No significant issues were identified during sensor testing. The modular wing design allowed individual sensor pairs to be repositioned and retested without affecting the rest of the circuit, which proved useful when verifying the 6cm mounting height required for reliable detection over the 5 cm maze walls.

8.4 Testing of Motor Drivers & Outcomes

The design of the motor circuits followed the standard solution that the team was provided with. Using this and the schematics that were created, the team first built one motor circuit to prove that the schematic was accurate. This was first done on a breadboard and tested using a power source and signal generator to simulate the steps. Once the breadboarded circuit successfully worked, it was recreated onto a Vero board and soldered. This was then also tested before the other motor circuit was soldered. Testing for the motors proved to be rather unstable with varying results. This was due to unstable connections and issues with the driver chip.

The team was able to successfully fine tune both motor circuits until the motors ran simultaneously, however this outcome was not always met despite no changes. This could possibly be due to bad contacts or soldering, however, was proven hard to fully identify and solve. They were resoldered more than once, and they would work for some time before torque issues appeared even though no changes or alterations were made after resoldering the verified circuit. The team was able to resolder the motor circuits to a working state for the prototype used for the competition.

8.5 Testing Micro-Mouse State Machine & Timing

This section describes the testing approach used to verify the correctness of the micro-mouse navigation state machine and the timing of all software-controlled operations. The state machine governs how the robot responds to sensor inputs and chooses its next movement on every iteration of the main loop. Timing verification ensures that motor step counts, sensor settling delays, and BFS recalculation all complete within acceptable durations for smooth, reliable cell by cell operation.

To make the state machine observable during development, the internal state variables: `mouse_position_x`, `mouse_position_y`, `orientation`, the `number_of_maze_cells` and `number_of_maze_walls` arrays were monitored by the MBED serial output during the software only testing on Keil Studio before hardware integration. By printing these values to the terminal at the start of each loop iteration, it was possible to trace the robot's decision-making step by step without requiring the physical robot, allowing logical errors in `choose_direction()`, `update_walls()`, and `share_walls()` to be identified and corrected early.

The state machine was verified using the test vectors shown in Table Below. Each test vector defines a combination of front, left, and right wall sensor inputs and specifies the expected navigation behaviour. These tests were designed to cover all junction types the robot can encounter in the maze.

Tests	Front	Left	Right	Expected Behaviour
Test 01	No	Yes	Yes	Move forward to lowest Lee number neighbour
Test 02	Yes	No	Yes	Turn right then move forward
Test 03	Yes	Yes	No	Turn left then move forward
Test 04	Yes	Yes	Yes	U-turn then move forward
Test 05	No	No	No	Move to lowest lees number neighbour (open intersection)
Test 06	Any	Any	Any	At [7][7] loop exits and robot stops

Several issues were raised and corrected because of this testing. The original code contained a function declaration in `main()` instead of a function call for `update_walls()`, meaning walls were never recorded at runtime identified when `number_of_maze_walls` remained all zeros throughout a simulated run. The `mouse_location()` function was also found to update position to diverge from the actual position over time. Both were corrected and retested to confirm expected behaviour. The `u_turn()` step count was also identified as identical to `turn_right()` at 400 steps, which would only produce a 90-degree rotation which was corrected to 800 steps.

Timing was verified by analysis of the `steps()` function with `wait_us(1000)` for both the high and low phase of each pulse, the step frequency is 500Hz. At 400 steps, each forward move or 90-degree turn takes approximately 1600ms. The `flood_fill_update()` BFS across all 64 cells executes in under 1ms on the 96MHz LPC1768 adding no meaningful delay to the loop cycle.

8.6 Testing of the Overall Micro-Mouse & Outcomes

Overall system testing was conducted using Keil Studio serial terminals as the primary observation tool, as full physical maze runs had not been completed at the time of writing. The integrated system was tested by manually supplying sensor input combinations through controlled test conditions and verifying the complete navigation pipeline sensor reading, wall mapping, flood fill update, direction selection, motor movement and position update executed correctly in sequence. The test vectors cover all junction types the robot can encounter in an 8x8 maze and verify both normal navigation and the goal-detection exit condition.

During integration testing, two previously identified software defects were confirmed as resolved. The `update_walls()` function, which had originally contained a function declaration rather than a function call inside `main()`, now correctly records all data into `number_of_maze_walls` on every loop iteration, verified by observing nonzero wall entries after the first read. The `u_turn(0)` step count correction from 400 to 800 steps was confirmed to produce a rotation consistent with 180 degrees, addressing the defect identified during state machine testing.

A third issue was identified during integration the `choose_direction()` function returned `direction -1` (no valid direction found) when all four neighbours of the current cell were blocked by walls during the initial flood fill run before any walls had been discovered. This was traced to the BFS in `flood_fill_setup()` not accounting for perimeter walls during the initial assignment, causing cells adjacent to the perimeter to receive a Lee number of 999 in some configurations. This was resolved by ensuring `setup_perimeter_walls()` always executes before the first BFS run, which is already enforced in the current code structure.

We did a full simulated maze run test where the robot manually stepped through a navigation loop with predefined wall inputs starting from `[0][0]` to `[7][7]` in the Keil Studio environment with manually supplied sensor inputs representing a simple maze configuration. The robot selected the correct direction at each step performing the appropriate turn move forward, updated its position and recalculated flood fill values before the next iteration. The goal cell was detected correctly, and the program exited the loop without error.

8.7 Evaluation of Test Results

Following the provided standard solution for the IR sensors to detect the walls, these were first individually produced and tested. Through this the team identified weaknesses within using this method. The IR transmitter and receiver were quite particular about angles and distance. There was also the issue of interference from neighbouring sensors. This was resolved by alternating the order of the IR transmitters and receivers on each wing. This significantly reduced interference and 3 pairs of IR sensors were able to function as expected on each wing. The addition of shielding was considered; however, it appeared to be unnecessary for this project.

The motor drivers were testing in a similar fashion to the sensors. By starting with the build of one motor and testing the functionality of it, weaknesses were quickly identified and errors were efficiently resolved. Once the team produced two individual motor circuits that worked, these were integrated together in order for them to move at the same speed.

By unit testing the individual circuits, then adding them to their respective sub system proved to be the most beneficial method of testing as each individual component was tested to prove for correct and expected functionality before being added to the main system. This allowed for easier identification of faults before being added to the overall circuitry. The limitation of this is that issues arose when components were being integrated together due to wrong wiring or loose soldering. With the number of small circuits being assembled together, the wiring became unorganized meaning it wasn't always easy to identify what part is causing a fault in the system.

Testing program verified all primary functions of the micro mouse software with the limitations of a simulation and bench testing environment; flood fill Lee algorithm, wall mapping, sensor integration, motor control and navigation state machine produces correct and specific results across all test vectors and conditions.

Bottom-up methodology testing proved effective by isolating and verifying each component and function before integration. Three significant defects were rectified through this testing, which were: missing `update_walls()` call, the divergent position tracking in mouse location `()` and the incorrect U-turn step count before they can disseminate into the full system. The serial terminal debugging provided sufficient visibility into the internal state to support this process without requiring additional hardware testing.

Once the hardware and software sides were individually tested and validated, they were integrated together by uploading the code onto the MBED on the micro mouse. The first main test done was to test if the micro mouse could drive down a straight line.

If so, this would indicate that the sensors are receiving data accurately and the code is processing and responding to it as intended. Although this did not happen smoothly during the first run, after a couple rounds of modifications and testing, the micro mouse was able to successfully turn in all directions and determine when to turn, go straight, or make a U-turn.

Section 9: Teamwork, Planning and Leadership

The teamwork in this project has been highly collaborative and effective in progressing in this project. Once we understood what members are in the team and our strengths and weaknesses, we assigned ourselves to what preference we had on whether to work more on the hardware side or software side of this project. Shortly after deciding roles, we got right to work by breaking down the different sub systems necessary to complete this project. the logic is that half of the team worked on building the physical micro mouse, while the other half worked on code for the different parts of this system.

Member	Primary Role/ Tasks	Contribution Rate
Suber	Aided in designing the structure of the code, produced code for all sensors, troubleshooted assembled micro mouse	20
Gabriel	Designed the motor driver circuits, produced the schematics, main motor tester for function and torque, tested circuits for shorts to validate functionality	20
Nicholas	Co-responsible for updating GitHub, produced code for mouse movement and decision making, tested code on the MBED	20
Charanjot	Co-responsible for updating GitHub, aided in testing code on the MBED, produced the maze algorithm code	15
Caleb	Built the front sensor, aided in hardware testing and troubleshooting	10
Victoria	Produced all wing sensors and tested them, assembled hardware components and aided in troubleshooting	15

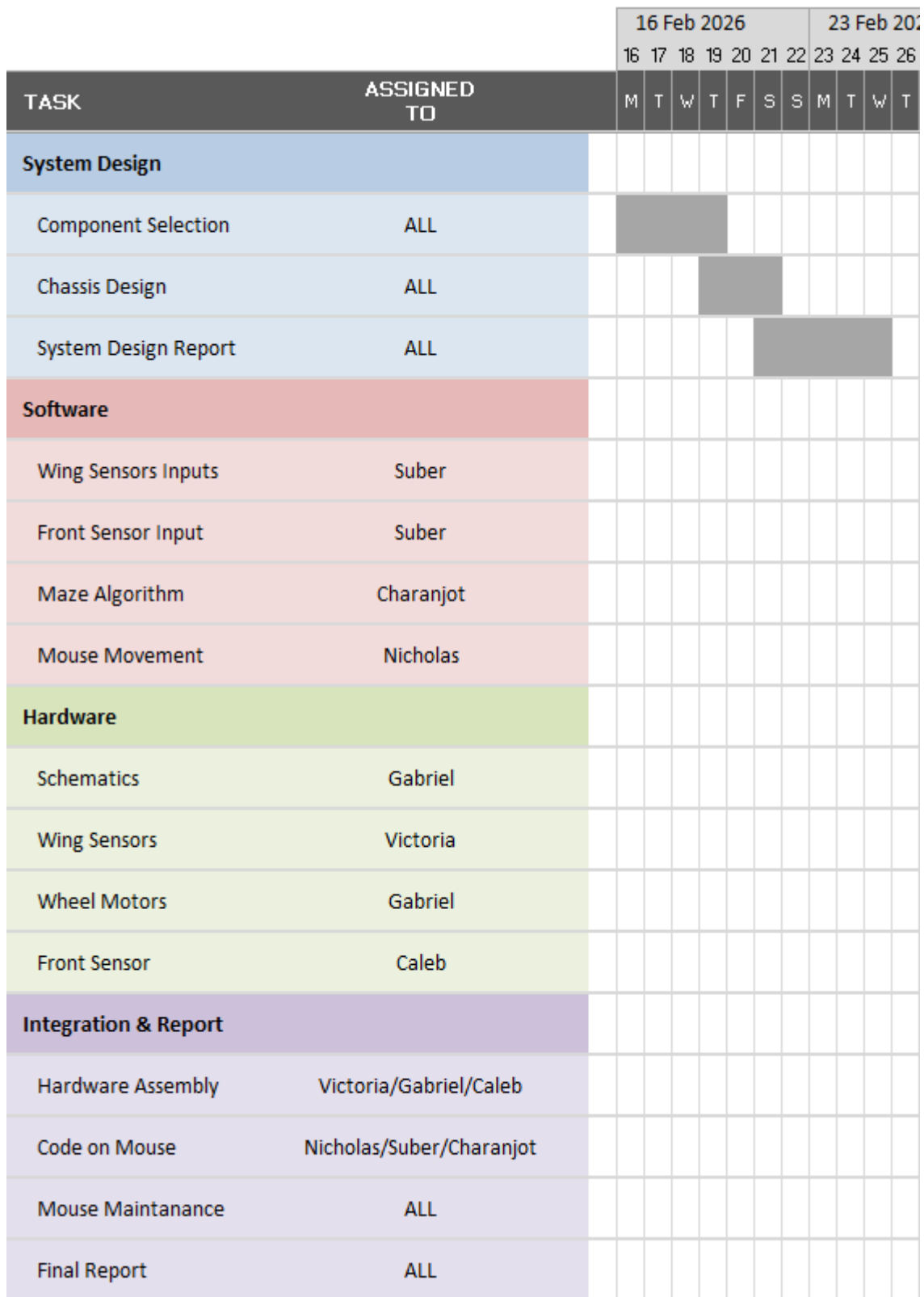


Figure 31: Gantt Chart 1/3

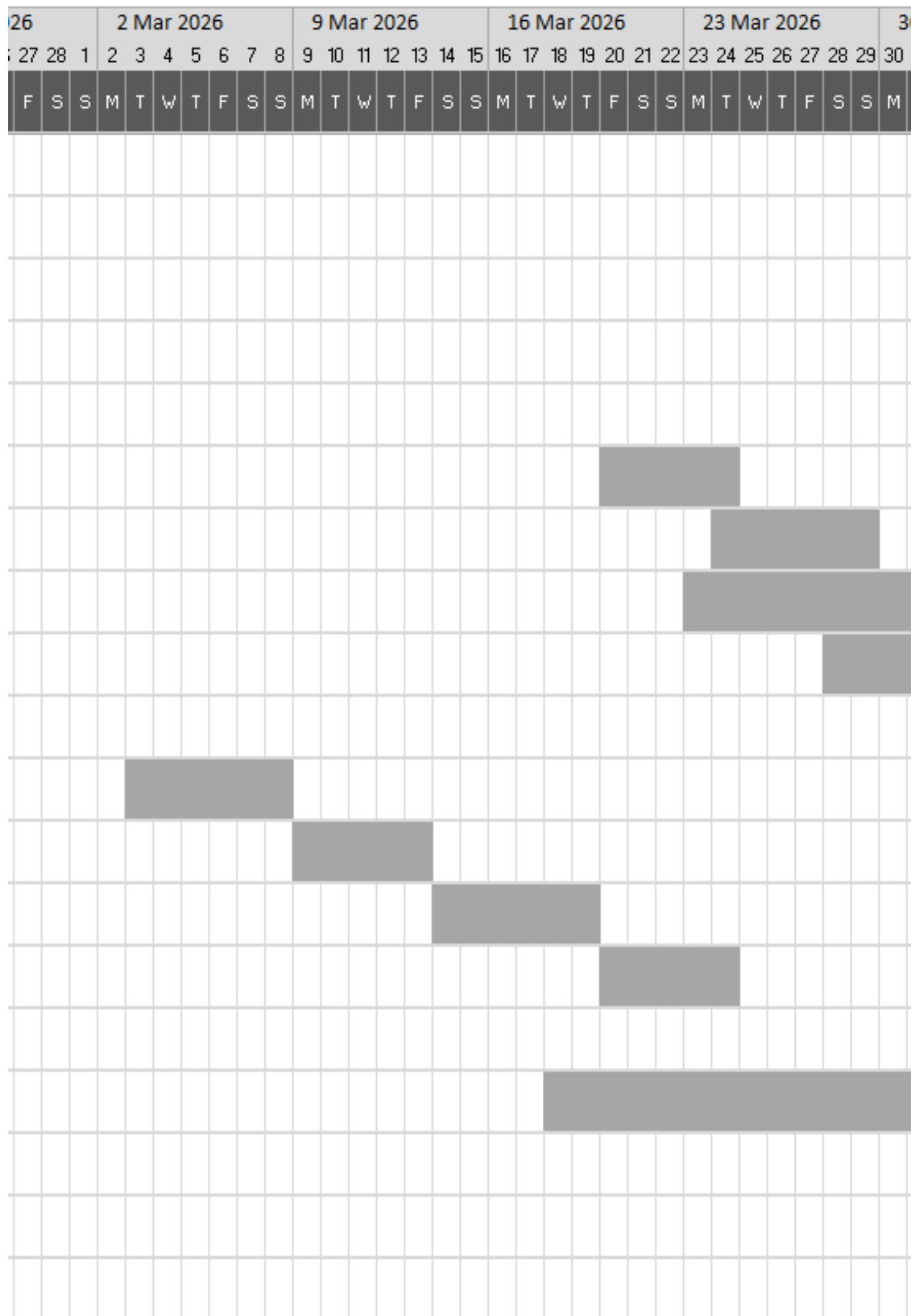


Figure 32: Gantt Chart 2/3

Figure 33: Gantt Chart 3/3

To stay on track and share what work had been done, we produced weekly meeting notes and video logs breaking down what we had achieved, problems we encountered and the goals for next week. Files were shared via a folder on One Drive and GitHub. A WhatsApp group was created for convenience to efficiently inform the team on individual progress and problems encountered allowing everyone to aid remotely if needed.

The team did not encounter any conflict and was able to mitigate tasks when team members were ill, had other commitments, or last-minute change of plans because of how we initially split the roles in the beginning of this project.

As tasks were allocated early on, leadership in the hardware and software side was needed to ensure a constant flow of progress to stay on track with the deadlines. Nicholas mainly led the software side of the project by setting a framework for the expected program to be created. Gabriel took the lead in hardware development by creating the schematics needed for connecting all the components together. Although this was a collaborative project, this does not mean each team member only completed one task. Support and aid were provided by all the members between tasks and sub systems when problems arose and were not straightforward to solve.

Conclusion

C.1 Achievements and Success

At the end of the project, we achieved having a volatile micro mouse which at times would successfully achieve movement and the sensors would be able to detect if any object was present, showing results through LEDs. However, it was incapable of navigating or movement through an 8 x 8 maze environment. However, during the short period of time, the robot was active and in working condition, it could detect walls and was functional in limited time for movement.

C.2 Component Choice and Future Changes

The choice to select infra-red sensors was optimal because of their fast response and short-range detection, making them effective for close objects (maze wall). However, they still have some weakness due to the external environment (e.g., ambient light) or interference (other IR sensors) that can affect accuracy. Ultrasonic sensors were selected due to long-range detection; it could see a wall from a long distance. An alternative approach to these sensors would be used more high precision IR sensors and using barriers to block the sensors from external factors and focus their emitting signal to a certain direction (lowering beam range/ or direction of infrared light is more concentrated).

Stepper motors were chosen over DC motors due to their accuracy which were needed precise 90-degree turns in a constrained cell. The choice came with both positives and negatives as the stepper motors are heavier and slower than DC motors causing issues to timings and balance of the micro mouse. For future iterations a DC motor with encoders could provide better results due to its balance between speed and precision as they are lighter, faster while being cheaper. The encoder would provide the needed accuracy for turning.

The chosen flood fill Lee's algorithm was selected as the navigational option due to its ability to dynamically reroute around walls it has discovered and always pursues the shortest path towards the goal. A suitable alternative would have been a left wall follow algorithm which would have been marginally easier to implement. This however would provide less optimal results as the algorithm tends to struggle with mazes where the goal is surrounded by an island configuration with no connected walls. Dijkstra's algorithm was also taken into consideration however was deemed unnecessary for a maze of our size (8x8) where flood fill BFS already produces optimal results at lower computational costs.

The chassis was 3D printed for this project which allowed for custom shape tailored to the component layout and sensor/wing positions. However, 3D printed PLA has limited structural rigidity under repeated motor vibration and is difficult to modify once printed. An aluminium chassis would offer superior durability and rigidity while it requires more manufacturing effort. It would better withstand the stresses of repeated maze runs. Aluminium is also more recyclable than PLA and is better in supporting the sustainability objectives discussed earlier in the report.

The modular Veroboard construction used across the project allowed individual sub-circuits to be built, tested and replaced independently. This proved highly beneficial during debugging. However, the distributed wiring between multiple boards introduced complexity and was a source of intermittent connection faults during integration. A custom printed circuit board would eliminate loose wiring, reduce the overall footprint of the electronics and produce a more reliable/repeatable result. This makes it the preferable choice for a production ready version of the robot.

C.3 Societal Impact and PLC

The design of our micro mouse meets several sustainability objectives: the use of NiMH batteries avoids toxic heavy metals, the Veroboard construction allows components to be replaced or rewired rather than discarding the entire system and socketed ICs such as the MBED and ULN2003 can be reused in future projects.

Across production, components were chosen with longevity in mind and breadboard prototyping allowed circuits to be tested before final assembly. This minimised material

waste during development. Regulatory compliance for a commercial version would require RoHS compliance, adherence to the WEEE Directive, and risk assessment under the Health and Safety at Work etc. Act 1974.

Robots based on these principles have both positive and negative societal impacts such as increased productivity and efficiency but also the displacement of workers. Accountability is a further issue as it is unclear whether the creator or the robot itself is at fault when errors occur. Cybersecurity is also a concern due to networked robots are vulnerable to being compromised by external actors.

C.4 Group Management

The group was managed through clear role allocation from the outset, with members assigned to either the hardware or software side based on individual strengths, allowing both to progress in parallel. Nicholas led the software side by establishing the initial code framework, while Gabriel led the hardware side by producing the schematics and overseeing motor driver development. Weekly meetings were documented through meeting notes and video logs.

Files were shared via OneDrive, GitHub and a WhatsApp group chat which was used for a day-to-day communication. No significant conflicts arose during the project where members were absent, tasks were redistributed without major disruption because roles had been clearly defined early and the modular structure of both the hardware and software meant individual contributions remained visible and accountable throughout.

C.5 Impact of Mentoring Sessions

The two mentoring sessions provided guidance that went beyond the technical scope of the project. One mentor had completed the same course at the same university and shared his personal experience tackling the micro mouse. This reassured the group that the integration challenges we encountered were a normal part of the process. The sessions covered the path to becoming a chartered engineer in detail. This included the requirements and process which developed the group's commercial awareness by framing the skills practised in this project systems design, testing, documentation and teamwork as directly relevant to professional engineering practice.

The mentors also asked each member about their future and offered tailored career advice. From a project management perspective, the sessions reinforced the value of structured planning and regular review points. This validated the approach the group had already adopted and helping the team stay focused on completing deliverables within the available time.

References

- [1] H. Yu, "Application of in DC Motor Control Method in Intelligent Vehicles and Robots," in *2022 IEEE 5th International Conference on Automation, Electronics and Electrical Engineering (AUTEEE)*, Shenyang, China, 2022, pp. 1018-1021, doi: 10.1109/AUTEEE56487.2022.9994475.
- [2] RS Components, "The Complete Guide to Stepper Motors," *RS Discovery Portal*, Jan. 06, 2023. [Online]. Available: <https://uk.rs-online.com/web/content/discovery/ideas-and-advice/stepper-motors-guide>. [Accessed: Apr. 24, 2026].
- [3] A. Williams, *Microcontroller Projects Using the Basic Stamp*, 2nd ed. San Francisco, CA, USA: CMP Books, 2002
- [4] Sharp Corporation, "IS471F: OPIC Light Detector with Built-in Signal Processing Circuit for Light Modulation System," Datasheet, [Online]. Available: <http://www.sigmaelectronics.net/manuals/IS471F.pdf>. [Accessed: Apr. 22, 2026].
- [5] Vishay Semiconductors, "TSOP344, TSOP348 Series: IR Receiver Modules for Remote Control Systems," Datasheet, Rev. 1.8, 2022. [Online]. Available: <https://uk.rs-online.com/web/p/ir-receivers/0564396>. [Accessed: Apr. 22, 2026].
- [6] TT Electronics/Optek Technology, "Infrared Emitting Diode: OP165, OP166 Series," Datasheet, 2023. [Online]. Available: <https://www.mouser.co.uk/ProductDetail/Optek-TT-Electronics/OP165D>. [Accessed: Apr. 26, 2026].
- [7] SparkFun Electronics, "Ultrasonic Distance Sensor - HC-SR04," Datasheet, 2019. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>. [Accessed: Apr. 29, 2026].
- [8] M. A. Oliver, "Maze-Solving Algorithms for Micromouse," *IEEE Robotics & Automation Magazine*, vol. 14, no. 3, pp. 22-26, 2007.
- [9] J. M. S. J. Baillie, "The Micromouse Maze Solving Contest," *Electronics & Wireless World*, vol. 92, no. 1604, pp. 75-78, 1986.
- [10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009, pp. 658-662.
- [11] D. Linden and T. B. Reddy, *Handbook of Batteries*, 3rd ed. New York, NY, USA: McGraw-Hill, 2002.

- [12] The Pi Hut, "8 x AA Battery Holder - Back to Back," Product Specification, 2024. [Online]. Available: <https://thepihut.com/products/8-x-aa-battery-holder>. [Accessed: Apr. 28, 2026].
- [13] Duracell, "Technical Bulletin: AA MN1500 Alkaline-Manganese Dioxide Battery," 2023. [Online]. Available: <https://www.duracell.co.uk>. [Accessed: Apr. 28, 2026].
- [14] Fairchild Semiconductor, "LM7805: 3-Terminal 1A Positive Voltage Regulator," Datasheet, 2014. [Online]. Available: <https://cdn.sparkfun.com/assets/1/7/7/3/2/LM7805.pdf>. [Accessed: Apr. 28, 2026].
- [15] STMicroelectronics, "L297: Stepper Motor Controller," Datasheet, 2001. [Online]. Available: <https://www.st.com/resource/en/datasheet/l297.pdf>. [Accessed: Apr. 28, 2026].
- [16] Texas Instruments, "ULN2003A High-Voltage High-Current Darlington Transistor Arrays," Datasheet, 2016. [Online]. Available: <https://www.ti.com/lit/ds/symlink/uln2003a.pdf>. [Accessed: Apr. 27, 2026].
- [17] Rich Electronics, "Texas ULN2003A Darlington Transistor Array Integrated Circuit 16P SOIC," Product Datasheet, 2024. [Online]. Available: <https://richelectronics.co.uk/product/texas-uln2003a-darlington-transistor-array-integrated-circuit-16p-soic-om035d>. [Accessed: Apr. 27, 2026].
- [18] Arm Mbed, "Mbed OS: Open-source embedded operating system for Arm Cortex-M," Documentation, 2024. [Online]. Available: <https://os.mbed.com/mbed-os/>. [Accessed: Apr. 27, 2026].
- [19] Arm Mbed, "mbed LPC1768: High-performance Arm Cortex-M3 microcontroller for prototyping," Platform Documentation, 2024. [Online]. Available: <https://os.mbed.com/platforms/mbed-LPC1768/>. [Accessed: Apr. 27, 2026].
- [20] IEEE, "IEEE Standard for Intelligent Robots and Systems," IEEE Std 1872-2015.
- [21] M. Dudek and M. Jenkin, *Computational Principles of Mobile Robotics*, Cambridge University Press, 2010.
- [22] S. Thrun, W. Burgard and D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- [23] International Federation of Robotics, "World Robotics Report 2023."
- [24] Amazon, "Amazon Robotics Overview," 2024.
- [25] UK Government, "Waste Electrical and Electronic Equipment (WEEE) Regulations," 2013.
- [26] European Commission, "RoHS Directive (Restriction of Hazardous Substances)," 2011.

[27] J. M. Allwood et al., *Sustainable Materials: With Both Eyes Open*, UIT Cambridge, 2012.

[28] Starship Technologies, “Autonomous Delivery Systems,” 2024.

[29] Institution of Engineering and Technology, “Code of Conduct,” 2021.

[30] UK Government, “UKCA Marking: Guidance for Manufacturers,” 2022.

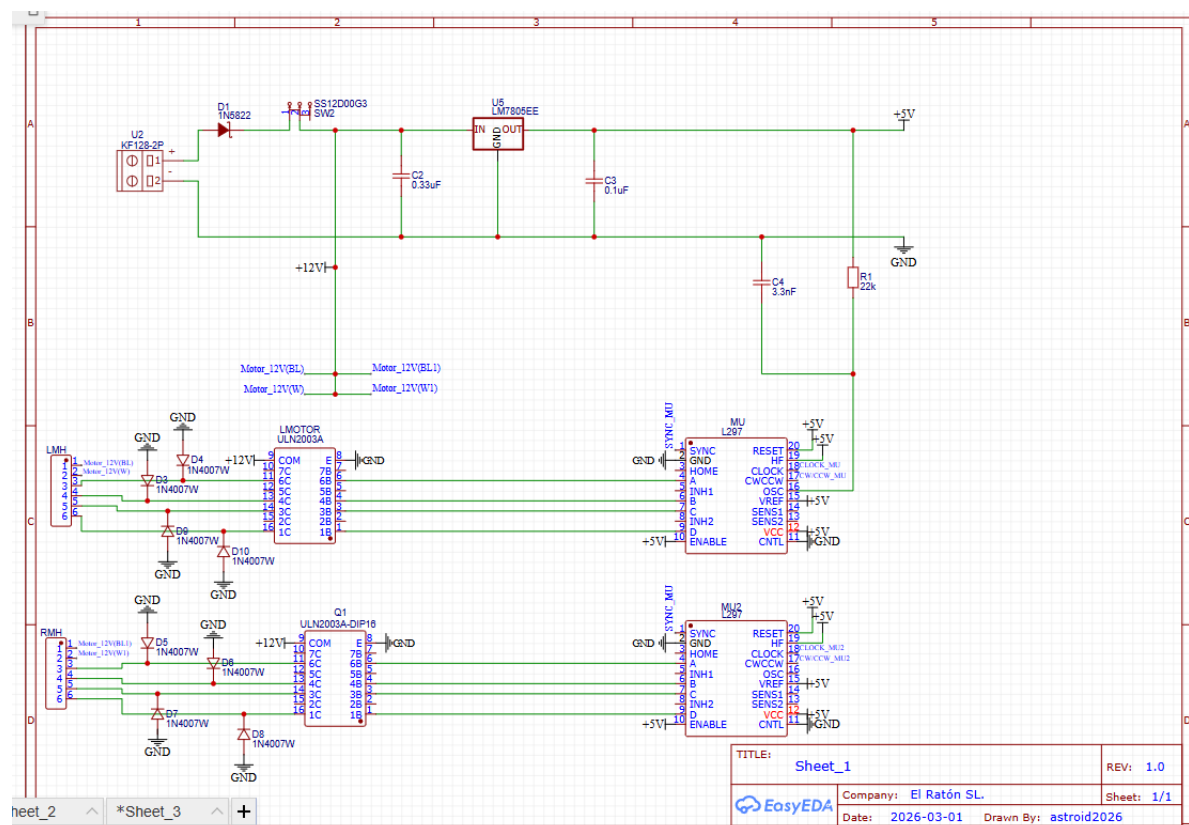
[31] UK GDPR, Data Protection Act 2018.

[32] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson, 2004.

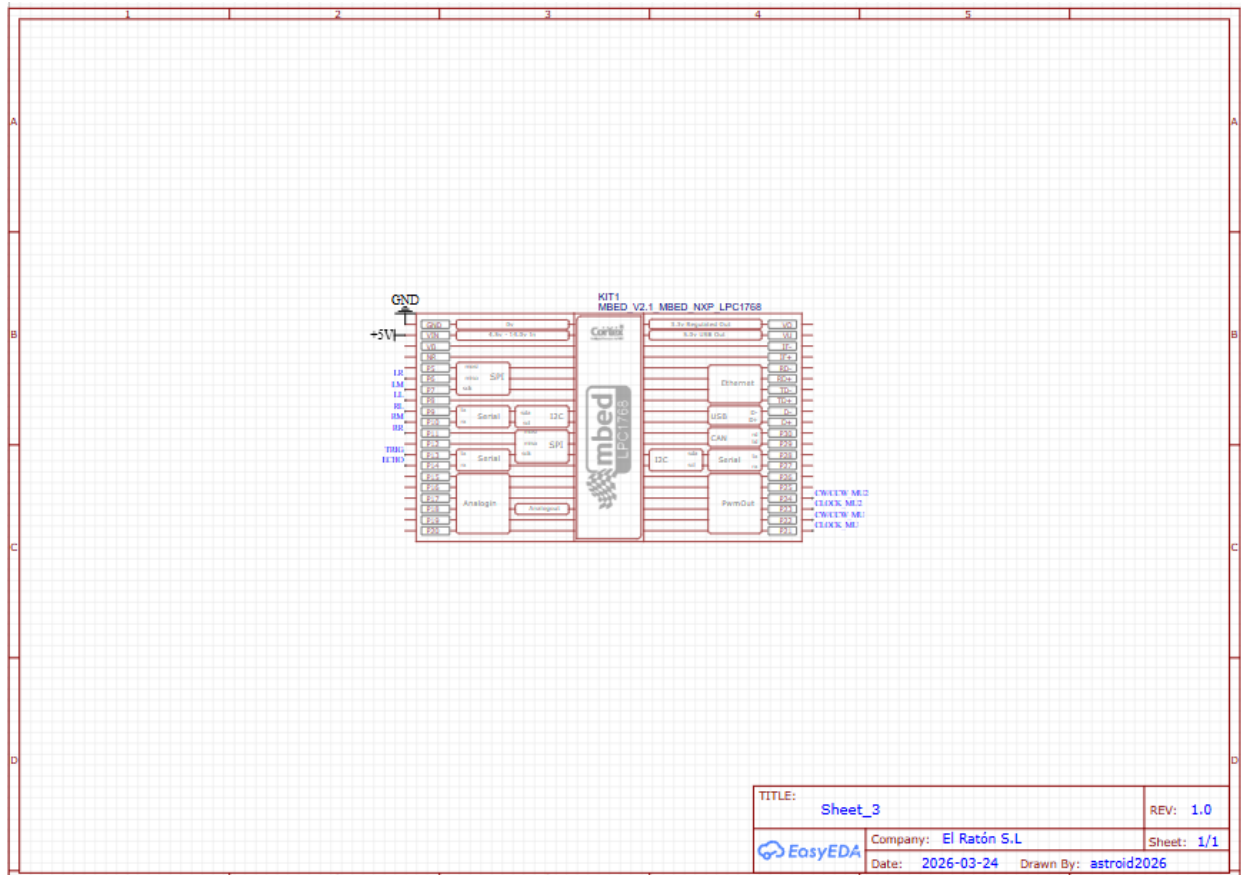
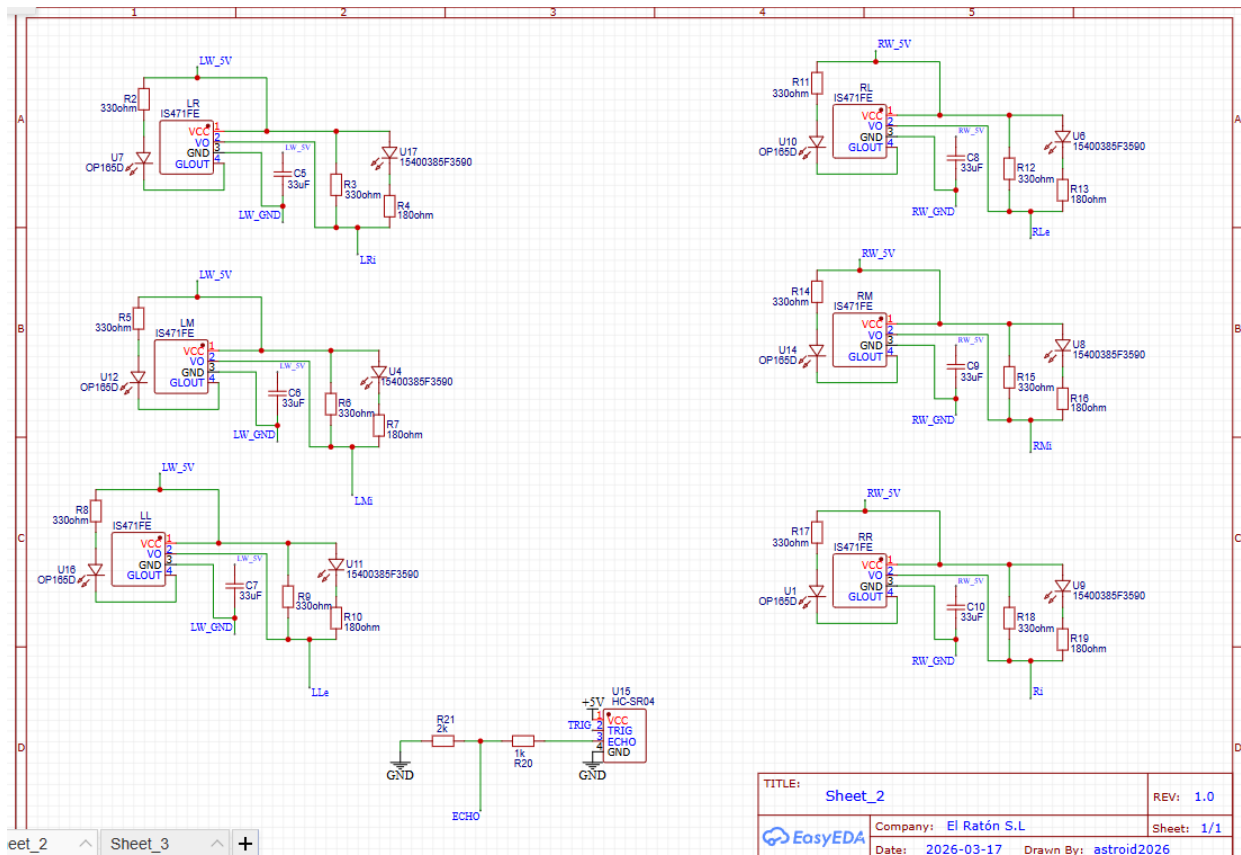
[33] Health and Safety Executive (HSE), “Safe Use of Electricity in the Workplace,” 2020.

[34] ISO, “ISO 12100: Safety of Machinery – General Principles for Design,” 2010.

Appendix A: Schematics



Group 3 | 5 ELEN020W.2 Embedded Systems Project (WBL) | 2026



Appendix B: Full Code

```
#include "mbed.h"

//Later change pins to right settings

DigitalOut Stepper_Motor_1_Step(p21); // Left Motor
DigitalOut Stepper_Motor_1_Direction(p22); // Left Motor
DigitalOut Stepper_Motor_2_Step(p23); // Right Motor
DigitalOut Stepper_Motor_2_Direction(p24); // Right Motor
DigitalIn US_Sensor_Echo(p12); // Ultrasonic Sensor
DigitalOut US_Sensor_Trigger(p13); // Ultrasonic Sensor
BusIn Left_Sensors (p8,p7,p6); // Left Bus Sensor
BusIn Right_Sensors (p11,p10,p9); // Right Bus Sensor

Timer t;

volatile int us_distance_mm = 0;

// North = 0 ,East = 1, South = 2, West = 3
int number_of_maze_walls[8][8];
int mouse_position_x = 0; // location of the mouse
int mouse_position_y = 0; // location of the mouse

int number_of_maze_cells[8][8];
int orientation = 0; // direction of the mouse, assume starting North always

void share_walls(int x, int y); // forward declaration
```

```
double check_us_distance(){
    uint32_t echo_time = 0; //supposed to have values range from 0 - 49000

    //Trigger pulse function
    US_Sensor_Trig = 0;
    wait_us(2);
    US_Sensor_Trig = 1;
    wait_us(10);
    US_Sensor_Trig = 0;
    //Wait for echo high
    while(!US_Sensor_Echo){};
    t.reset();
    t.start();

    //Wait for echo low
    while(US_Sensor_Echo){
    }
    t.stop();

    echo_time = t.elapsed_time().count();

    //Speed of sound = 0.343 mm/us
    us_distance_mm = (echo_time*343)/2000;
    if(us_distance_mm < 80){
        t.stop();
    }
    return (double)us_distance_mm;
}
```

```
int check_left_sensors(){  
    int left_wall_value = Left_Sensors.read();  
    if (left_wall_value == 7){  
        return 1;  
    }  
    // can add elseif for alignment issues but need hardware to test  
    else {  
        return 0;  
    }  
}
```

```
int check_right_sensors(){  
    int right_wall_value = Right_Sensors.read();  
    if (right_wall_value == 7){  
        return 1;  
    }  
    // can add elseif for alignment issues but need hardware to test  
    else {  
        return 0;  
    }  
}
```

```
bool wall_in_front(){  
    return check_us_distance() < 80;  
}
```

```
bool wall_to_left(){  
    return check_left_sensors() == 1;
```

```
}

bool wall_to_right(){
    return check_right_sensors() == 1;
}

void update_walls(int orientation, bool front, bool left, bool right){
    if (orientation == 0){
        if (front) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 1; // Mark
        North Wall, Front is North (1)

        if (left) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 8; // Mark
        West Wall, Left is West (8)

        if (right) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 2; // Mark
        East Wall, Right is East (2)
    }
    else if (orientation == 1){
        if (front) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 2; //
        Front is East (2)

        if (left) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 1; // Left is
        North (1)

        if (right) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 4; // Right
        is South (4)
    }
    else if (orientation == 2){
        if (front) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 4; //
        Front is South (4)

        if (left) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 2; // Left is
        East (2)

        if (right) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 8; // Right
        is West (8)
    }
    else if (orientation == 3){
```

```
    if (front) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 8; //
Front is West (8)

    if (left) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 4; // Left is
South (4)

    if (right) number_of_maze_walls[mouse_position_x][mouse_position_y] |= 1; // Right
is North (1)

}

share_walls(mouse_position_x, mouse_position_y); //share walls with neighbours
}
```

```
void share_walls(int x, int y){

    // If this cell has a North wall, the cell above gets a South wall
    if((number_of_maze_walls[x][y] & 1) && y < 7)

        number_of_maze_walls[x][y+1] |= 4;

    // If this cell has an East wall, the cell to the right gets a West wall
    if((number_of_maze_walls[x][y] & 2) && x < 7)

        number_of_maze_walls[x+1][y] |= 8;

    // If this cell has a South wall, the cell below gets a North wall
    if((number_of_maze_walls[x][y] & 4) && y > 0)

        number_of_maze_walls[x][y-1] |= 1;

    // If this cell has a West wall, the cell to the left gets an East wall
    if((number_of_maze_walls[x][y] & 8) && x > 0)

        number_of_maze_walls[x-1][y] |= 2;

}
```

```
void setup_perimeter_walls(){

    for(int i = 0; i < 8; i++){

        number_of_maze_walls[i][0] |= 4; // South wall on entire bottom row

        number_of_maze_walls[i][7] |= 1; // North wall on entire top row

    }

}
```

```
    number_of_maze_walls[0][i] |= 8; // West wall on entire left column
    number_of_maze_walls[7][i] |= 2; // East wall on entire right column
}
}

void flood_fill_setup(){
    // Initialise all cells to 0
    for(int x = 0; x < 8; x++){
        for(int y = 0; y < 8; y++){
            number_of_maze_cells[x][y] = 0;
        }
    }

    setup_perimeter_walls(); //load outer walls before BFS runs so it doesnt go past outer
    walls

    bool visited[8][8] = {};

    // Stores cells in queue
    int queue_x[64], queue_y[64];
    int head = 0, tail = 0;

    number_of_maze_cells[7][7] = 0;
    visited[7][7] = true;
    queue_x[tail] = 7; queue_y[tail] = 7; tail++;

    // Process till empty
    while(head != tail){
        int cx = queue_x[head];
        int cy = queue_y[head];
```

```
head++;

int next_val = number_of_maze_cells[cx][cy] + 1;

// Check North neighbour, if not visited assign Lee number and add to queue
if(cy < 7 && !visited[cx][cy+1]){
    number_of_maze_cells[cx][cy+1] = next_val;
    visited[cx][cy+1] = true;
    queue_x[tail] = cx; queue_y[tail] = cy+1; tail++;
}

// Check East neighbour
if(cx < 7 && !visited[cx+1][cy]){
    number_of_maze_cells[cx+1][cy] = next_val;
    visited[cx+1][cy] = true;
    queue_x[tail] = cx+1; queue_y[tail] = cy; tail++;
}

// Check South
if(cy > 0 && !visited[cx][cy-1]){
    number_of_maze_cells[cx][cy-1] = next_val;
    visited[cx][cy-1] = true;
    queue_x[tail] = cx; queue_y[tail] = cy-1; tail++;
}

// Check West
if(cx > 0 && !visited[cx-1][cy]){
    number_of_maze_cells[cx-1][cy] = next_val;
    visited[cx-1][cy] = true;
    queue_x[tail] = cx-1; queue_y[tail] = cy; tail++;
}
```

```
}  
}
```

```
void flood_fill_update(){  
    // Reset all cells, walls will block propagation as they are discovered  
    for(int x = 0; x < 8; x++){  
        for(int y = 0; y < 8; y++){  
            number_of_maze_cells[x][y] = 999;  
        }  
    }  
}
```

```
    // Set goal cell [7][7] to 0 and add to queue  
    number_of_maze_cells[7][7] = 0;  
    int queue_x[64], queue_y[64];  
    int head = 0, tail = 0;  
    queue_x[tail] = 7; queue_y[tail] = 7; tail++;
```

```
    // Process queue until empty  
    while(head != tail){  
        int cx = queue_x[head];  
        int cy = queue_y[head];  
        head++;  
  
        int next_val = number_of_maze_cells[cx][cy] + 1;  
  
        // Check North - skip if north wall exists  
        if(cy < 7 && !(number_of_maze_walls[cx][cy] & 1) &&  
            number_of_maze_cells[cx][cy+1] == 999){
```

```
    number_of_maze_cells[cx][cy+1] = next_val;

    queue_x[tail] = cx; queue_y[tail] = cy+1; tail++;

}

// Check East

if(cx < 7 && !(number_of_maze_walls[cx][cy] & 2) &&
number_of_maze_cells[cx+1][cy] == 999){

    number_of_maze_cells[cx+1][cy] = next_val;

    queue_x[tail] = cx+1; queue_y[tail] = cy; tail++;

}

// Check South

if(cy > 0 && !(number_of_maze_walls[cx][cy] & 4) && number_of_maze_cells[cx][cy-
1] == 999){

    number_of_maze_cells[cx][cy-1] = next_val;

    queue_x[tail] = cx; queue_y[tail] = cy-1; tail++;

}

// Check West

if(cx > 0 && !(number_of_maze_walls[cx][cy] & 8) && number_of_maze_cells[cx-
1][cy] == 999){

    number_of_maze_cells[cx-1][cy] = next_val;

    queue_x[tail] = cx-1; queue_y[tail] = cy; tail++;

}

}

}
```

```
void steps(){ // expand on loops when testing hardware
```

```
    Stepper_Motor_1_Step = 1;
```

```
    Stepper_Motor_2_Step = 1;
```

```
    wait_us(1000);
```

```
    Stepper_Motor_1_Step = 0;
```

```
Stepper_Motor_2_Step = 0;
wait_us(1000);
}
```

```
void forwards(){
    Stepper_Motor_1_Direction = 1;
    Stepper_Motor_2_Direction = 1;
    for (int i = 0; i < 400; i++){
        steps();
    }
}
```

```
void turn_left(){
    Stepper_Motor_1_Direction = 0;
    Stepper_Motor_2_Direction = 1;
    for (int i = 0; i < 400; i++){
        steps();
    }
}
```

```
void turn_right(){
    Stepper_Motor_1_Direction = 1;
    Stepper_Motor_2_Direction = 0;
    for (int i = 0; i < 400; i++){
        steps();
    }
}
```

```
void u_turn(){

    //u_turn is now double the steps of turn_right to rotate 180 degrees

    Stepper_Motor_1_Direction = 1;
    Stepper_Motor_2_Direction = 0;
    for (int i = 0; i < 800; i++){
        steps();
    }
}

//choose best direction using flood fill Lee numbers
int choose_direction(){
    int best_val = 9999;
    int best_dir = -1; // N=0, E=1, S=2, W=3

    // Check North - no north wall and in bounds
    if(mouse_position_y < 7
    && !(number_of_maze_walls[mouse_position_x][mouse_position_y] & 1)){
        if(number_of_maze_cells[mouse_position_x][mouse_position_y+1] < best_val){
            best_val = number_of_maze_cells[mouse_position_x][mouse_position_y+1];
            best_dir = 0;
        }
    }

    // Check East
    if(mouse_position_x < 7
    && !(number_of_maze_walls[mouse_position_x][mouse_position_y] & 2)){
        if(number_of_maze_cells[mouse_position_x+1][mouse_position_y] < best_val){
            best_val = number_of_maze_cells[mouse_position_x+1][mouse_position_y];
            best_dir = 1;
        }
    }
}
```

```
    }
}

// Check South
if(mouse_position_y > 0
&& !(number_of_maze_walls[mouse_position_x][mouse_position_y] & 4)){
    if(number_of_maze_cells[mouse_position_x][mouse_position_y-1] < best_val){
        best_val = number_of_maze_cells[mouse_position_x][mouse_position_y-1];
        best_dir = 2;
    }
}

// Check West
if(mouse_position_x > 0
&& !(number_of_maze_walls[mouse_position_x][mouse_position_y] & 8)){
    if(number_of_maze_cells[mouse_position_x-1][mouse_position_y] < best_val){
        best_val = number_of_maze_cells[mouse_position_x-1][mouse_position_y];
        best_dir = 3;
    }
}

return best_dir;
}

void clamp_position(){ //forces position values to stay in the range
    if(mouse_position_x < 0) mouse_position_x = 0; //this shows if x goes wrong like -1 this
    snaps it back to 0
    if(mouse_position_x > 7) mouse_position_x = 7;
    if(mouse_position_y < 0) mouse_position_y = 0;
    if(mouse_position_y > 7) mouse_position_y = 7; //8 or higher snaps back to 7
}
```

```
int main(){

    flood_fill_setup();

    while(1){
        wait_us(10000);

        bool front = wall_in_front();

        bool left = wall_to_left();

        bool right = wall_to_right();

        wait_us(10000);

        update_walls(orientation, front, left, right);

        wait_us(10000);

        // Check if goal reached

        if((mouse_position_x == 7) && (mouse_position_y == 7)){

            break;

        }

        //use flood fill to pick best direction

        int target_dir = choose_direction();

        // Work out how many right turns needed to face target direction

        int turns_needed = (target_dir - orientation + 4) % 4;

        if(turns_needed == 0){

            // Already facing correct direction, move forward

            forwards();

        }

    }

}
```

```
else if(turns_needed == 1){
    turn_right();
    forwards(); // turn then move forward into the cell
}
else if(turns_needed == 2){
    u_turn();
    forwards();
}
else if(turns_needed == 3){
    turn_left();
    forwards();
}

// Update orientation and position after moving
orientation = target_dir;
if(orientation == 0) mouse_position_y++;
else if(orientation == 1) mouse_position_x++;
else if(orientation == 2) mouse_position_y--;
else if(orientation == 3) mouse_position_x--;

clamp_position();

flood_fill_update();
}
}
```